

Agentic Visual Generation: From Generative Models to Agentic Control

Yinming Huang^{*}, Shuyuan Tu^{*}, Xi Yan^{*}, Jiahao Zhan, Zihan Yang, Zhen Xing, Hui Zhang, Tiehua Zhang, *Member, IEEE*, Yu-Gang Jiang, *Fellow, IEEE*, and Zuxuan Wu[†], *Member, IEEE*

How Agentic Control Evolves:

Agentic control progressively expands across the generation process, from shaping inputs to adapting future decisions. As autonomy increases, human supervision recedes.

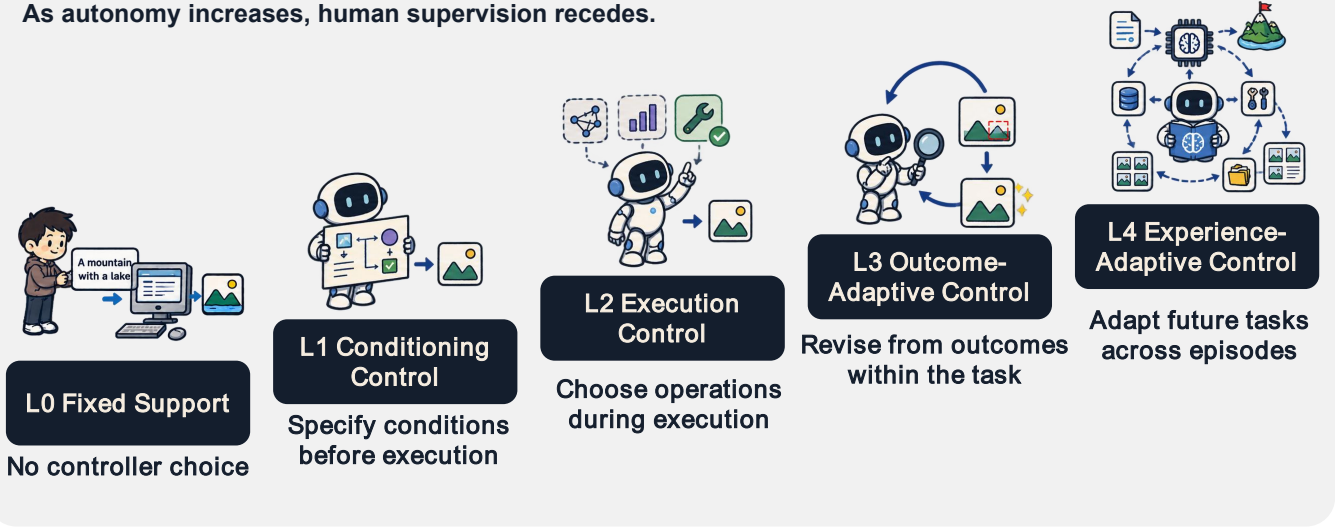


Fig. 1. Visual overview of controller decision-making scope. L0 Fixed Support supplies predetermined generative components. L1 Conditioning Control constructs a specification for a predetermined executor. L2 Execution Control selects and invokes visual operations. L3 Outcome-Adaptive Control uses observed outcomes to revise the current task. L4 Experience-Adaptive Control retains completed-task experience that changes a later independent task.

Abstract—Visual generation is evolving from generative models used through a single invocation into agentic control processes that can plan, select tools, inspect intermediate synthesized outputs, revise failures, and reuse prior experience. In most existing systems, the controller is an LLM or VLM, while visual generation models serve as tools or executors. However, existing work lacks a consistent criterion for determining when a generation system becomes agentic. Planning depth, tool use, multi-role collaboration, and reinforcement learning are often treated as evidence of agenticity, even though none of them necessarily determines which generation decisions the controller can make. We organize the field according to what the controller can directly control in the generation process. At L1 Conditioning Control, the controller prepares the input to a predetermined generator but does not control which visual operation is executed.

At L2 Execution Control, it selects and invokes actual generation, editing, rendering, or other content-modifying operations. At L3 Outcome-Adaptive Control, it observes an intermediate outcome and uses that observation to change a subsequent operation within the current task. At L4 Experience-Adaptive Control, it retains experience from completed tasks and uses that experience to change decisions on future tasks. L0 Fixed Support separately denotes generators, editors, evaluators, reward models, benchmarks, and fixed pipelines without a deployed controller that makes generation-level decisions. These levels describe a progressively broader decision-making scope rather than model size, system complexity, output quality, tool or role count, or training method. Applying this framework across image, video, editing, 3D, world, slide, and user-interface generation reveals how controller capabilities have evolved and how their mechanisms are distributed across levels. We further develop a level-conditioned evaluation framework that isolates the value of broadening the controller’s decision-making scope by matching generators, tools, budgets, and evaluators across levels. Finally, we identify the key transitions from generative models to executable control, reliable outcome-driven revision, reusable cross-task experience, and a future generator-as-controller regime. Project resources are available at the project repository.

Y. Huang, S. Tu, X. Yan, Z. Yang, H. Zhang, Z. Wu, and Y-G. Jiang are with Fudan University. Emails: {ymhuang26, sytu23, 24307140086, yangzh26, hui_zhang23}@m.fudan.edu.cn, {zxwu, ygj}@fudan.edu.cn

Y. Huang and Z. Wu are also with Shanghai Innovative Institute.

J. Zhan is with CUHK, MMLab. Email: 1155271220@link.cuhk.edu.hk.

Z. Xing is with Wan Team, Alibaba Tongyi Lab. Email: zx-ing20@fudan.edu.cn.

T. Zhang is with the School of Computer Science and Technology, Tongji University. Emails: tiehuaz@tongji.edu.cn.

^{*}These authors contributed equally. [†]Corresponding author.

Index Terms—Agentic visual generation, image generation,

video generation, world model, reinforcement learning.

I. INTRODUCTION

VISUAL generation, including image, video, structured visual content, and 3D generation, has long been a challenging research problem that supports creative applications, communication, software development, and simulation. Deep generative models have substantially improved visual quality and language alignment. DALL-E 2 [1], Imagen [2], and Parti [3] established strong text-conditioned image synthesis at scale. Latent Diffusion [4], SDXL [5], and DALL-E 3 [6] subsequently improved efficient training, high-resolution generation, and instruction following. Video Diffusion Models [7] and Make-A-Video [8] transferred these advances to temporal synthesis, while Imagen Video [9], Video LDM [10], and Lumiere [11] further developed high-resolution, latent-space, and space-time generation designs [12]. These advances improve what a visual executor can produce from a supplied condition [13]. In our hierarchy, these fixed generators, editors, retrievers, evaluators, and predetermined pipelines constitute **L0 Fixed Support**. L0 marks the inclusion boundary because these components provide generation and evidence capabilities but do not contain a deployed controller that decides how the generation process should proceed.

The transition from L0 Fixed Support to **L1 Conditioning Control** addresses a limitation of a fixed executor: the user's request may not directly provide a usable spatial specification or the external knowledge needed for generation. L1 methods solve this problem by constructing the condition consumed by one predetermined executor. LLM-grounded Diffusion (LMD) [14] converts a complex request into object descriptions and bounding boxes that guide a frozen diffusion model. LayoutGPT [15] similarly uses in-context reasoning to produce explicit 2D or 3D layouts before rendering. World-To-Image [16] retrieves definitions and reference images when the generator lacks knowledge of a requested entity, then incorporates that evidence into the generator-facing condition. These methods therefore resolve ambiguity before generation. Their remaining limitation is that the controller still cannot decide which visual operation should execute.

The transition from L1 Conditioning Control to **L2 Execution Control** addresses this operation-selection limitation. An L2 controller can choose and invoke a generator, editor, program, or workflow rather than only prepare the input to a fixed executor. Visual ChatGPT [17] turns visual foundation models into callable tools and uses a language-model controller to select and invoke the operation required by the request. ComfyUI-Copilot [18] constructs an executable node graph whose components and data flow determine the generation route. ViMax [19] extends executable control to coordinated video operations such as script preparation, shot planning, character styling, and clip generation. These systems solve the problem of selecting and sequencing capabilities, but their selected route can remain open loop because a generated result does not necessarily change the next action.

The transition from L2 Execution Control to **L3 Outcome-Adaptive Control** addresses failures that become visible only

after execution. Benchmarks such as T2I-CompBench [20] and GenEval [21] reveal compositional and object-binding errors, while VBench [22] and EvalCrafter [23] expose temporal and perceptual defects in generated videos. These evaluators diagnose the limitation, but they do not solve it by themselves. L3 systems close the loop by mapping an observed result to a later generation action. SLD [24] converts a diagnosed mismatch into a new sampling decision, while GenPilot [25] uses visual feedback to choose a subsequent refinement. L3 therefore extends control beyond execution, although the resulting state and repair experience can remain confined to the current task.

The transition from L3 Outcome-Adaptive Control to **L4 Experience-Adaptive Control** addresses this episode boundary. An L4 controller retains information from a completed task and uses it to change decisions on a later independent task. OctoT2I [26] updates persistent capability profiles that inform future routing decisions. GenEvolve [27] distills successful and failed generation trajectories into reusable procedures, while COMFYCLAW [28] promotes verified workflow-construction procedures into a reusable skill library. Together, L0 through L4 are the five labels in our hierarchy. The progression has a specific causal meaning: each transition extends the latest point at which evidence can change a future generation decision, from constructing a condition at L1, to selecting an operation at L2, revising the current task at L3, and adapting future tasks at L4. A future generator-as-controller is therefore not introduced as an additional level. It is classified by the same maximum-causal-reach test.

The literature nevertheless remains fragmented across tasks and uses inconsistent criteria for identifying agenticity. A unified account must distinguish controller decision-making scope from generator progress, implementation topology, and learning procedure.

Scope, classification, and evaluation. Against this background, we develop a controller-centered framework that places fixed supporting components in L0 Fixed Support, which marks the inclusion boundary, and classifies L1–L4 systems by the latest future generation decision their controller can change. This single test applies across modular systems, multi-role workflows, and unified generative policies. It also determines the matched evaluation design: each comparison adds one class of controller decisions while holding the generator, tools, budget, and evaluator fixed.

Inclusion boundary. This definition also determines the inclusion boundary of the framework. Every system within our principal scope must contain a primary visual generator or editor controlled by a generation-level decision process. The system may collaborate with other agents or call external tools. A standalone supporting module or a fixed pipeline is not treated as an agentic visual generation system by itself.

Contributions. The contributions are as follows:

- We define agenticity by the maximum temporal and causal reach of the decisions a controller can make. This criterion yields a reproducible hierarchy from L0 Fixed Support through L4 Experience-Adaptive Control. L0 marks the inclusion boundary, while a category and subcategory taxonomy organizes systems by the object controlled and its technical realization.

- We release a structured corpus with level, task, mechanism, feedback, memory, resource, and provenance fields. Its temporal and cross-sectional statistics reveal the rapid shift toward within-trajectory feedback while persistent cross-task experience remains uncommon.
- We organize representative systems as a design space of controlled variables and feedback paths, emphasizing the causal decisions that distinguish methods rather than enumerating paper titles.
- We propose level-conditioned evaluation that isolates the value of specification, execution, outcome feedback, and reusable experience under matched generators, tools, budgets, and evaluators.

Organization. Sections II–III define the generation–control distinction and the L0–L4 taxonomy. Sections IV–VIII analyze L0–L4, Section IX covers training, Section X develops matched evaluation, and Sections XI–XII discuss open transitions and conclude the paper.

II. FROM VISUAL GENERATION TO GENERATION-LEVEL CONTROL

Visual generation describes what an executor can create or edit. Agentic visual generation additionally describes how a controller selects conditions, invokes operations, and changes later actions around that executor. This section establishes that distinction, defines the controller and generator roles, and introduces the task and mechanism axes used in the paper.

A. Conceptual Foundations: Agents and Visual Generation

This separation begins with the two concepts that the field often conflates. Agentic visual generation combines a controller with the state-construction capability of a visual generator. Separating these functions is important because a model that accepts rich conditions does not automatically control a generation trajectory, and a controller that only interprets images is outside the generation scope of this work.

1) *Agents:* We first characterize the controller side of this separation. An agent is a goal-directed system that selects actions from observations and maintains enough state to adapt later decisions. Its controller can be a language model, a multimodal language model, a learned policy, a search procedure, or a team of specialized roles. The defining property is not the controller architecture. It is the presence of intermediate decisions that influence task execution. In visual generation, these decisions include prompt revision, layout construction, model routing, reference selection, editing, verification, memory update, and stopping.

Agentic behavior can be implemented by multiple coordinated models or by a single model [29], [30]. A multi-model system may combine a controller with specialized generators, editors, or evaluators. A single-model system, such as a unified multimodal model, may perform several of these functions through one learned policy. In both cases, agenticity requires an action space, an observation process, and a mechanism through which current state or feedback changes a later generation decision. Model count does not determine agenticity.

2) *Visual Generation:* The generator side supplies the complementary state-construction capability. Visual generation covers the creation, editing [31], [32], and refinement of images [33], videos [34]–[38], visual stories, slides, user interfaces, 3D scenes [39], and interactive environments. For slides and user interfaces, the action representation may be PowerPoint objects, XML, HTML, CSS, or executable code, but the synthesized content is still judged through its rendered visual structure and behavior. Latent Diffusion [4], Video Diffusion Models [7], and DreamFusion [40] learn mappings from conditions to pixels, temporal sequences, or radiance fields. Design2Code [41] instead predicts executable interface representations. Generation controllers place these mappings within a larger decision process. Their generated content is not only a terminal output. It may also be an observation, a memory item, a candidate for comparison, or a persistent world state.

B. Definition of Agentic Visual Generation

With the controller and generator roles separated, the next question is how they must interact for a complete system to enter the scope of this work. We define an agentic visual generation system as a system that contains a visual generator or editor and a decision process that controls generation over one or more steps. Let g denote the user goal, s_t the current multimodal state, a_t an action, and o_{t+1} the resulting observation. The controller follows $a_t \sim \pi(a_t \mid g, s_t)$, while the environment transition $s_{t+1} = F(s_t, a_t, o_{t+1})$ may include a newly generated image, video segment, scene, critique, or retrieved reference. The objective is not only to maximize visual quality. It may balance instruction satisfaction, consistency, controllability, latency, monetary cost, and human effort.

This formulation highlights three properties. First, the system creates part of its own future observation space. Second, the action space can mix symbolic actions, tool calls, and visual generation. Third, the quality of a trajectory depends on both the final output and the decisions used to obtain it.

C. Basic Components

The definition specifies controller decision-making scope abstractly, so we next identify the components that realize it. Every system in our scope contains both a *controller* and a *primary visual generator or editor*. The generator creates or updates the synthesized visual content, while the controller maps the goal, state, and observations to generation-level actions. In the predominant architecture, an LLM, VLM, or MLLM serves as this controller, while visual generators and renderers are exposed as tools or executors. The following paragraphs distinguish distributed roles and tools, single-model controllers, and the optional memory or learning mechanisms that can extend either design. A generator does not become an agent merely by producing complex visual content.

Collaborating roles and tools. The controller may collaborate with specialized agents or invoke external tools. Collaborating agents can interpret user intent, construct plans, assign subtasks, critique intermediate results, preserve consistency, or manage long-horizon workflows. External tools

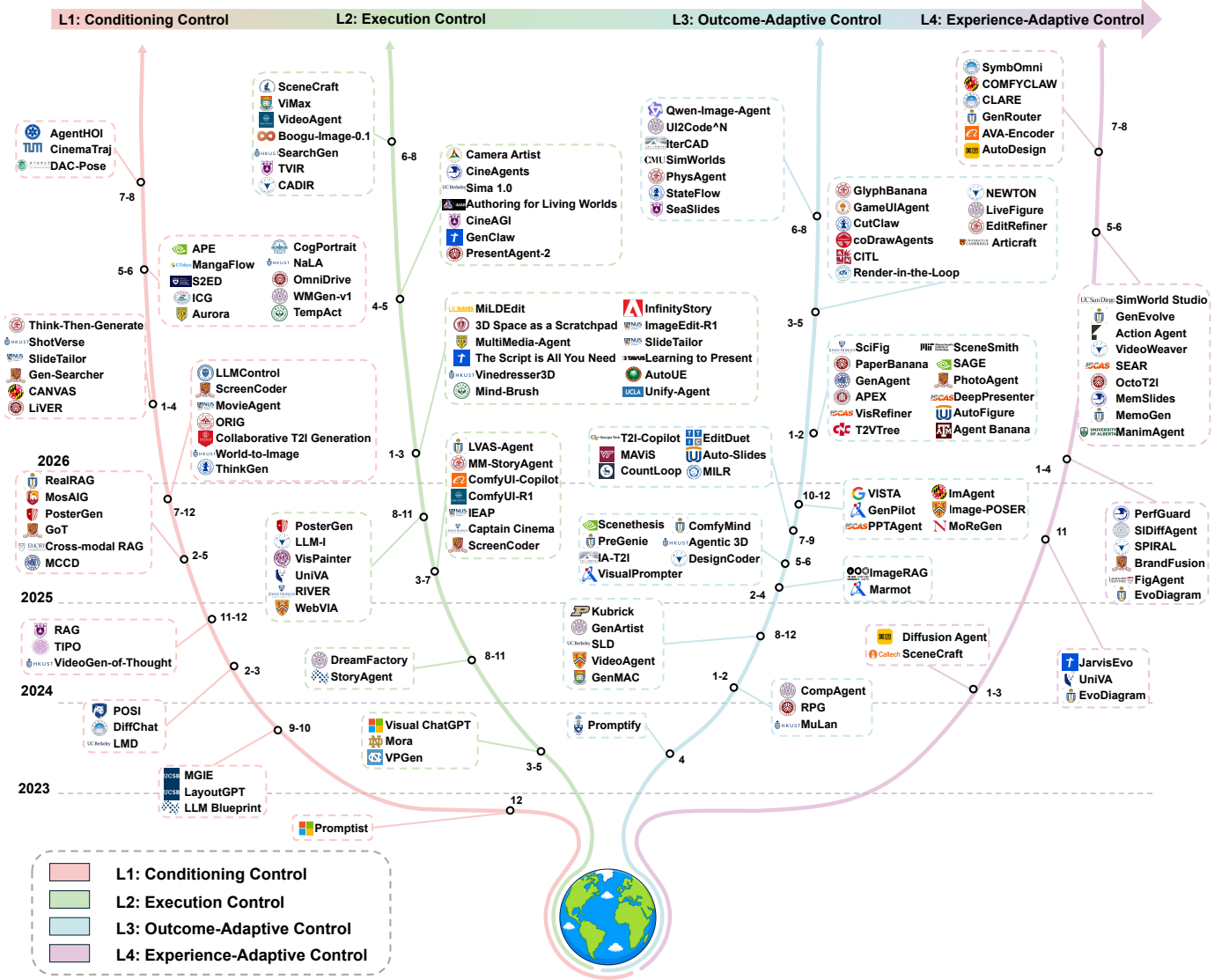


Fig. 2. Popular-paper roadmap across the L1–L4 controller levels, organized by first public release and the latest decision capability exposed by each representative system, from condition construction and operation selection to outcome-driven revision and reusable cross-task experience; the roadmap is a qualitative overview rather than a quantitative ranking.

can include retrievers, controlled generators, editors, detectors, segmenters, renderers, simulators, PowerPoint object models, browser runtimes, verifiers, and reward models. Their outputs provide conditions, executable actions, evidence, or feedback that the controller uses to decide how the synthesized content should be created or revised.

Single-model controllers. When these functions are not distributed across roles and tools, agentic behavior can instead be implemented by one learned model. Unified multimodal models are the main example in the current literature. UI2Code^N [42], for instance, generates synthesized content, inspects its rendered state, and decides whether to refine it. We treat a single model as a controller only when it demonstrates a control decision over conditions, execution, or later actions. Joint understanding and generation, one-shot visual-token prediction, or reward-based generator post-training alone remain insufficient. Within the analyzed corpus, most systems

still combine a language-based or multimodal language-based controller with a separate visual generator.

Memory and learning. Regardless of whether control is distributed or implemented by one model, memory and learning can further retain accepted synthesized outputs, user preferences, world states, tool experience, and successful strategies. These mechanisms are optional for open-loop and within-episode agents, but become defining capabilities when information from completed tasks changes later control. This persistence criterion also applies to a future generator-as-controller system, which would need to select tools or models, manage state, interpret outcomes, and alter its own trajectory rather than only update pixels or visual tokens.

D. Task Axis

Components describe what a system contains, but comparison also requires separating what is generated from how

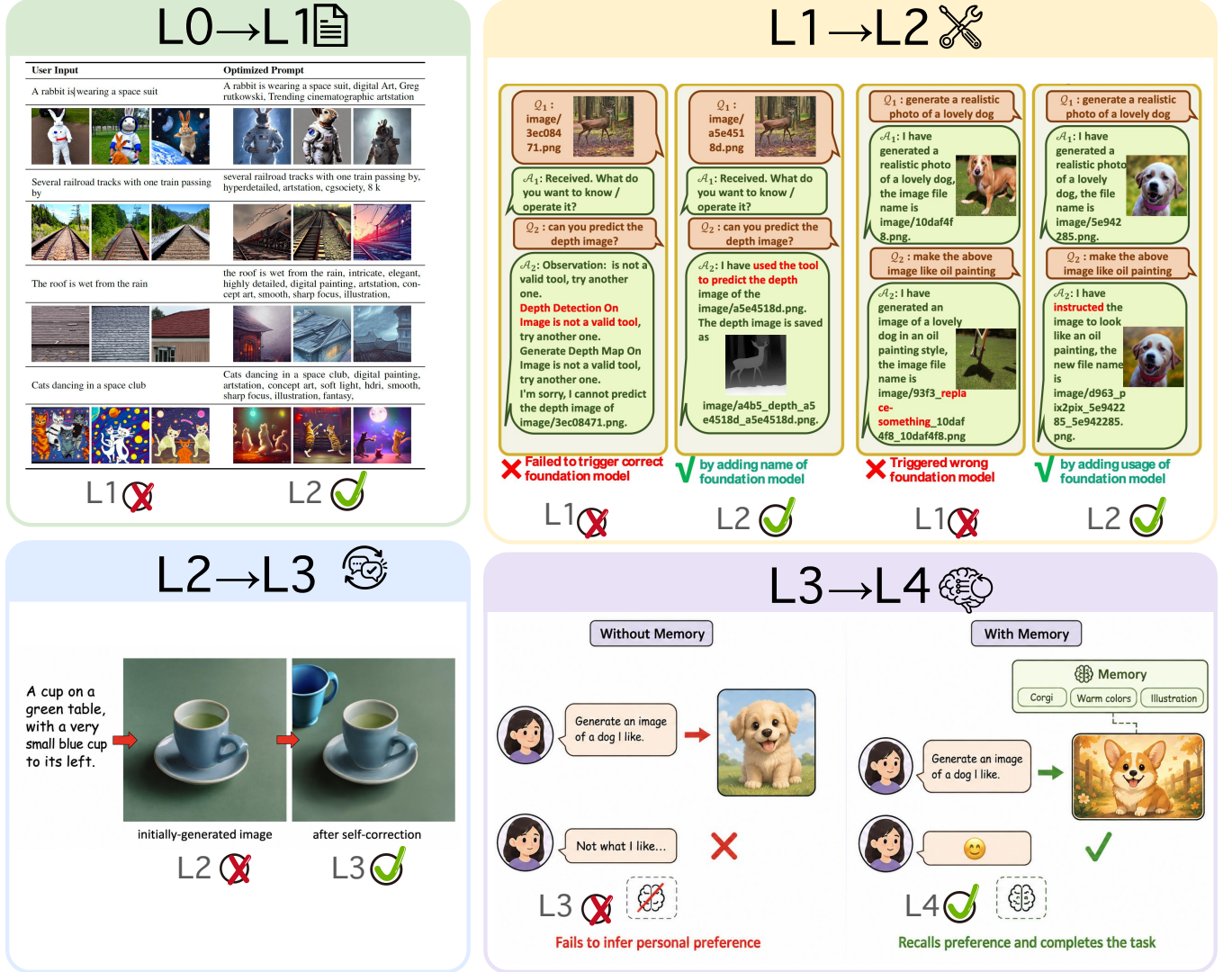


Fig. 3. Illustrative comparisons across adjacent controller levels. Each panel adds one decision capability to the preceding level: condition construction in L1 Conditioning Control, operation selection in L2 Execution Control, outcome-dependent revision in L3 Outcome-Adaptive Control, or cross-task experience reuse in L4 Experience-Adaptive Control. The examples are conceptual and do not constitute quantitative comparisons.

generation is controlled. The task axis includes image generation and editing, video generation and editing, slide and user-interface generation, 3D asset and scene construction, world-grounded synthesis, and interactive simulation. These tasks differ in temporal horizon, state persistence, action granularity, and the cost of evaluating intermediate results. Task type does not determine controller capability. An image system can exhibit a longer adaptive trajectory than a video system, while a multi-shot video pipeline can remain open loop.

E. Mechanism Axis

Because task type does not reveal controller decision-making scope, a second axis records the mechanism used to realize it. The mechanism axis includes intent grounding, explicit or latent planning, tool routing, retrieval, multi-agent collaboration, verification, memory, and reinforcement learning. These mechanisms describe how a controller is

implemented, but they do not define its level. Tool use specifies an action space, multi-agent design specifies a topology, and reinforcement learning specifies an optimization procedure. Any of them can appear at several capability levels.

III. A HIERARCHY OF CONTROLLER DECISION-MAKING SCOPE

The task and mechanism axes describe a system, but neither orders agenticity. Our organizing principle is instead causal: *agenticity in visual generation is determined by the deepest point in a generation trajectory at which a controller can causally change a future generation decision*. “Deepest” refers to temporal reach along the trajectory. A condition precedes execution, an execution decision determines which visual operation occurs, an observed outcome can redirect a later action within the same task, and persistent experience can affect an action after the current task has ended.

A. Controller-Capability Axis

We classify each complete system by the latest future generation decision that its controller can change. Table I is the canonical definition of the five labels. Lower capabilities remain visible as a capability path, while the primary label records only the maximum demonstrated reach. For example, L1+L2+L3 records specification construction, operation invocation, and outcome-dependent revision, while assigning the system to L3 Outcome-Adaptive Control. The following paragraphs first establish L0 Fixed Support and its boundary role, then separate decision-making scope from architecture, and finally state the decision procedure.

L0 Fixed Support. L0 is not a peer agent level. It records fixed supporting components and marks the inclusion boundary. A generator, editor, retriever, evaluator, reward model, benchmark, or fixed pipeline can be essential to an agent without selecting generation-level actions itself. Separating this boundary prevents complexity and support quality from being mistaken for controller decision-making scope.

Decision-making scope rather than architecture. With L0 Fixed Support established, the next distinction separates capability from implementation because common architectural labels collapse causally different systems. Several planners may still terminate in one declarative specification, whereas a compact router can directly determine which generator or editor executes. Likewise, a dynamically assembled workflow can remain open loop: its route may vary by request without changing after a generated result is observed. The hierarchy therefore records the controller's decision-making scope, not implementation complexity, model size, output quality, tool or role count, or training procedure.

Decision procedure. These distinctions lead to an assignment rule. We apply the tests from highest to lowest. Cross-task persistence establishes L4 Experience-Adaptive Control; an outcome-to-action causal link establishes L3 Outcome-Adaptive Control; selection and invocation of a visual operation establishes L2 Execution Control; and specification construction for a predetermined executor establishes L1 Conditioning Control. If none holds, the method is assigned to L0 Fixed Support. Ambiguous cases receive the lower level until the paper demonstrates the missing causal link. This rule makes the taxonomy reproducible and prevents terminology in a title, such as "multi-agent," "self-reflective," or "self-evolving," from serving as evidence by itself.

B. Fine-Grained Taxonomy and Roadmap

The controller-capability level identifies the latest decision that a system can change, but it does not by itself organize systems that act at the same level. We therefore use one explicit second-stage axis per level. L0 is organized by support function, L1 by the primary generator-facing specification, L2 by the primary executable object, L3 by the decisive feedback source, and L4 by the persistent experience carrier. A **subcategory** captures a stable distinction within that axis rather than an implementation architecture or a label created for one paper. Each system receives one primary placement within its level; secondary capabilities remain part of the

prose comparison. Figure 4 gives the complete category-to-subcategory roadmap.

C. Why a Hierarchy Rather Than Independent Capability Dimensions?

Planning, tool use, feedback, and memory are useful annotations, but treating them as independent categories does not answer which system can intervene later in the causal chain. Let a system have a capability vector

$$\mathbf{c}(S) = (c_1, c_2, c_3, c_4), \quad c_i \in \{0, 1\}, \quad (1)$$

where the four entries denote demonstrated control over specification, execution, within-task outcome adaptation, and cross-task experience reuse. We retain this vector as the capability path and define the primary level as

$$L(S) = \max\{i : c_i = 1\}. \quad (2)$$

Two properties justify this projection: it orders systems by maximum causal reach and applies the same test across visual modalities.

Maximum causal reach. The hierarchy is therefore a projection of a richer design space onto maximum causal reach. Each successive level extends the controller's causal reach to a later decision in the generation trajectory, while output quality and implementation complexity remain separate evaluation dimensions. It makes one comparison legible: how late can evidence still change a future generation decision?

Modality independence. Beyond ordering causal reach, the same projection remains stable across modalities. A storyboard planner and a spatial-layout planner can both be L1 because their decision-making scope terminates at a specification. An image router and a 3D asset selector can both be L2 because they invoke different visual operations. The representation changes, but the causal test does not.

D. Boundary Cases and Conservative Assignment

Table II applies the decision test to cases that are often mislabeled. The primary level follows the demonstrated inference-time decision-making scope, not the language used in a paper title. When evidence for a causal link is absent, we assign the lower level.

Two distinctions are especially important. First, within-task memory is ordinary trajectory state and cannot establish L4. Second, reinforcement learning can optimize a generator, router, repair policy, or memory policy. Its level follows the learned policy's inference-time action space and causal reach, not the optimizer.

E. Quantitative Landscape of the Analyzed Systems

The hierarchy becomes more informative when applied consistently at corpus scale. We deduplicated systems from seven task collections and annotated first release date, primary level, capability path, modality, mechanism, controller type, visual executor, feedback boundary, evaluation type, cross-task persistence, and public resources. The complete CSV and JSON records, generation script, and summary statistics are

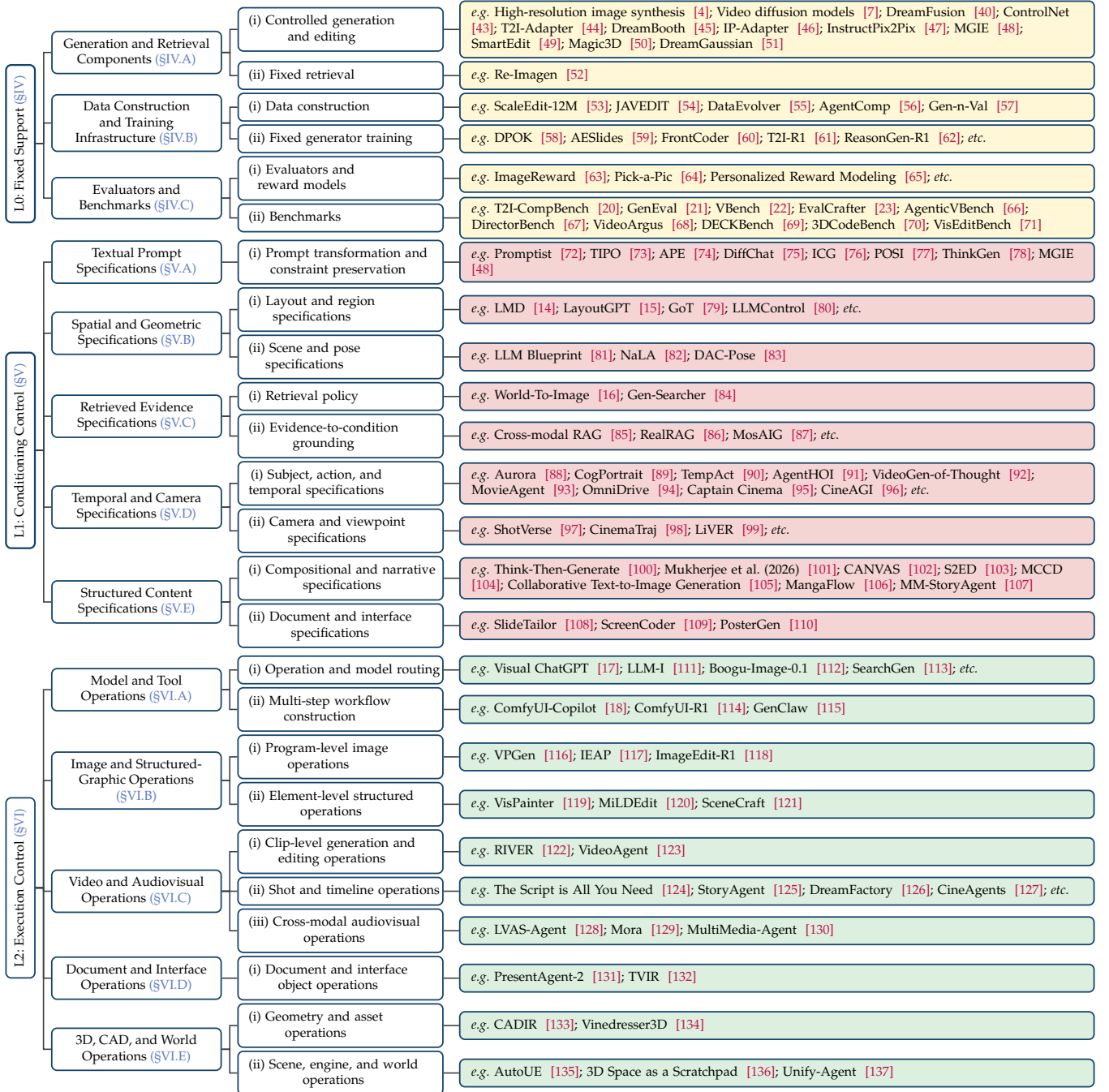


Fig. 4. Four-tier taxonomy and roadmap of controller decision-making scope. Part I covers L0–L2.

released with the project repository.¹ Counts below are generated from the same README records rather than transcribed into separate spreadsheets.

1) *Capability Evolution Over Time*: Figure 5 replaces a single total-count curve with level composition. The corpus begins with sparse L1 Conditioning Control, while the steep growth after 2025 is dominated by L3 Outcome-Adaptive Control. Only four reviewed records are assigned to L0 Fixed Support, because the corpus was assembled around controller-bearing systems and retains L0 cases only when they

resolve an important classification ambiguity. Recent work increasingly treats rendered outputs, execution results, and verifier diagnoses as state for subsequent decisions. In contrast, L4 Experience-Adaptive Control remains a small fraction of the literature, suggesting that persistent experience reuse is substantially less mature than within-task correction.

2) *Task and Controller Organization*: Temporal growth does not imply uniform adoption across tasks. Figure 6(a) shows that L3 is prevalent across all modalities, although its operational meaning varies by task. Editing and user-interface tasks expose rendered states that can be inspected and revised, whereas video, 3D, and world tasks require state consistency

¹Structured corpus: <https://github.com/YinmingHuang/Awesome-agentic-visual-generation-model>

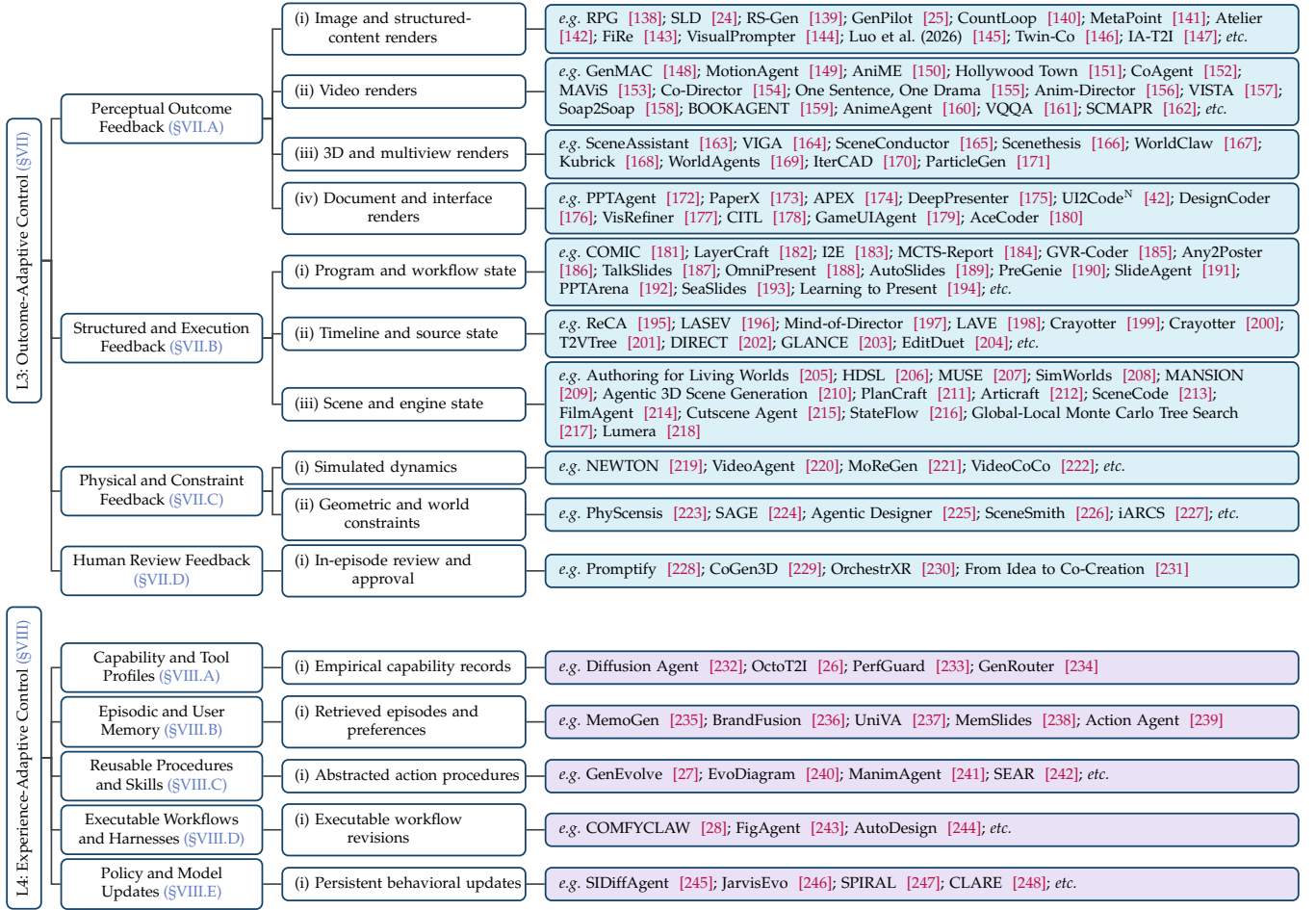


Fig. 4. Four-tier taxonomy and roadmap of controller decision-making scope (continued). Part II covers L3–L4.

TABLE I
CONTROLLER-CAPABILITY TAXONOMY OF AGENTIC VISUAL GENERATION

Level	Scope	Test question	Controlled variables	Status
L0	Fixed Support	Is the inference path predetermined?	Generator, retriever, evaluator, benchmark	Inclusion boundary
L1	Conditioning Control	What specification reaches a fixed executor?	Prompt, layout, reference, knowledge, storyboard	Controller level
L2	Execution Control	Which visual operation is invoked?	Tool/model, mode, program, call order	Controller level
L3	Outcome-Adaptive Control	What action follows an observed result?	Repair, reroute, regenerate, rollback, stop	Controller level
L4	Experience-Adaptive Control	What changes on the next task?	Memory, skills, profiles, policy, workflow library	Controller level

across time or viewpoints. Image generation has the largest absolute count and also contains most L1 controllers based on prompt construction, layout planning, and retrieval.

Controller organization. Figure 6(b) uses one architectural axis instead of mixing capabilities, topology, and training methods. Each system receives exactly one primary organization label: a single language or multimodal controller, a multi-role controller, or a unified multimodal policy. Single-controller systems dominate every level. Multi-role organizations occur mainly from L1 through L3, while unified policies remain less common but appear at every controller

level. Reinforcement learning is omitted from this comparison because it is an optimization procedure rather than a controller organization and is analyzed separately in Section IX.

Implications. These observations support two conclusions that guide the rest of the paper. First, outcome-conditioned revision is currently the most common form of agentic behavior. Its evaluation must therefore distinguish genuine outcome-driven repair from repeated sampling followed by selection, where apparent improvement may come from the selector rather than from adaptive revision. Second, the scarcity of cross-task persistence makes the transition from L3 to L4 an

TABLE II
BOUNDARY CASES UNDER THE MAXIMUM-CAUSAL-REACH RULE

Behavior	Level	Decisive test
Prompt rewrite + fixed generator	L1	Scope ends at the declarative specification
Generator/editor choice	L2	Controller selects the operation that executes
Critique + repair/regeneration	L3	Outcome changes a later action in the same task
Current-task state only	$\leq L3$	State does not cross the episode boundary
Reusable skill from a completed task	L4	Experience changes a later task's control
RL-trained fixed generator	L0/L1	Level follows inference-time decisions, not optimization
Multi-planner storyboard + fixed executor	L1	Role count does not extend causal reach
Candidate scoring without follow-up action	L0	Evaluation provides evidence only

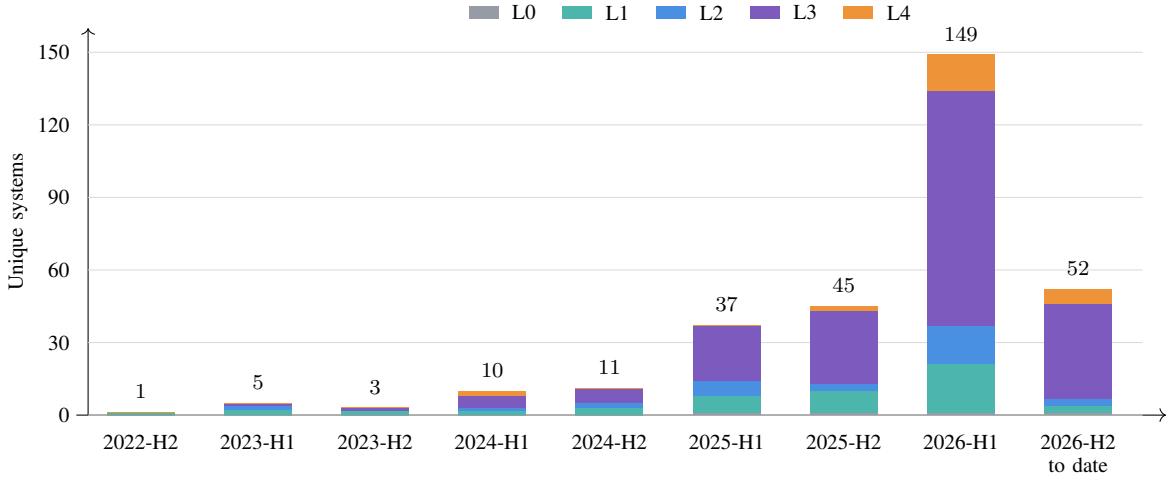


Fig. 5. Half-year capability evolution in the structured corpus, grouped by first public release. The 2026-H2 bar contains 52 records released through August 24, 2026. Unlike a total-only trend, the stacked bars expose which kind of controller decision drives growth.

(a) Modality $\times$ primary level						(b) Controller organization $\times$ primary level					
	L0	L1	L2	L3	L4		L0	L1	L2	L3	L4
Image	2	30	19	107	15	Single controller	4	42	25	163	23
Editing	1	4	13	51	4						
Video	1	16	10	42	6	Multi-role controller	0	6	6	28	0
Slide	1	1	1	11	1						
UI	0	1	0	7	0	Unified multimodal policy	0	1	2	11	2
3D	0	2	4	33	2						
World	1	3	3	13	2						

Fig. 6. Corpus cross-sections. Panel (a) is multi-label because one system may span generation and editing or multiple output types. Panel (b) assigns each system to exactly one primary controller organization. Cell shading is normalized within each row.

open research challenge rather than a routine extension. The following sections therefore examine not only what each level can do, but also what evidence is required to establish its additional causal reach.

IV. L0: FIXED SUPPORT

L0 Fixed Support supplies components to later controllers and marks the inclusion boundary rather than a peer agent category. We organize these components by their support function: generation or retrieval, data or training, and eval-

uation. As Figure 7 shows, a supplied condition c follows the predetermined inference path

$$y = G(c), \quad (3)$$

through a fixed generator or editor G . Table III summarizes the three parallel support functions. The final boundary test is a decision rule rather than a fourth component category.

A. Generation and Retrieval Components

L0 begins with fixed operations that later controllers may invoke. Latent Diffusion Models [4], Video Diffusion Mod-

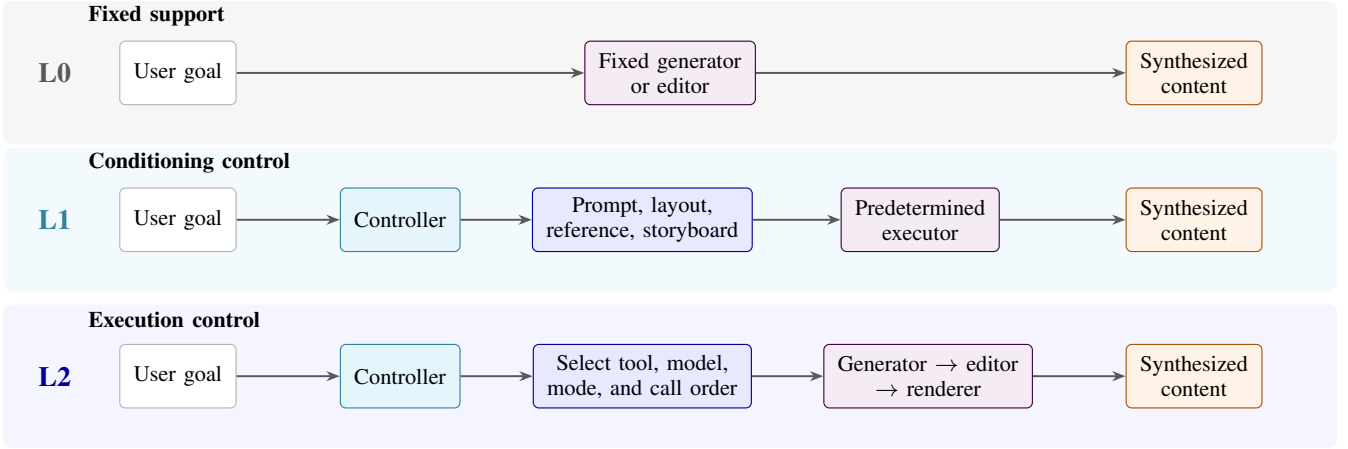


Fig. 7. From L0 Fixed Support to L2 Execution Control. L0 components such as ControlNet [43] execute a supplied condition along a predetermined path. L1 Conditioning Control systems such as LayoutGPT [15] construct a generator-facing specification. L2 Execution Control systems such as Visual ChatGPT [17] and ComfyUI-Copilot [18] select and invoke actual visual operations, but remain open-loop when the resulting synthesized content does not redirect the chosen route.

TABLE III
FINE-GRAINED ORGANIZATION OF L0 FIXED SUPPORT

Category	Subcategory	Controlled decision	Representative systems
Generation/retrieval	Controlled generation and editing	Execute a supplied condition	Latent Diffusion [4], Video Diffusion [7], ControlNet [43], InstructPix2Pix [47]
	Fixed retrieval	Follow a predetermined retrieval path	Re-Imagen [52]
Data/training	Data construction	Construct or filter records	AgentComp [56], Gen-n-Val [57], ScaleEdit-12M [53]
	Fixed generator training	Update parameters under fixed inference	DPOK [58], AeSlides [59], OSPO [249]
Evaluation	Evaluators and reward models	Score quality or validity	ImageReward [63], GenEval [21], VBench [22]
	Benchmarks	Diagnose capability or failure	AgenticVBench [66], DECKBench [69], 3DCodeBench [70]

els [7], and DreamFusion [40] map supplied conditions to image, video, and 3D representations. They establish what can be executed, but not which operation should be selected in a task. We separate these primitives into controlled generation and editing, which transform a supplied condition, and fixed retrieval, which supplies evidence through a fixed path.

Controlled generation and editing. ControlNet [43] and T2I-Adapter [44] execute spatial conditions, while DreamBooth [45] and IP-Adapter [46] support subject or reference conditioning. InstructPix2Pix [47], MGIE [48], and SmartEdit [49] similarly expose instruction-based editing operations. Recent controllable generators broaden this fixed-operation interface: CreatiLayout [250] controls layout-to-image synthesis, CreatiDesign [251] unifies image, layout, and text conditions for graphic design, MagicMotion [252] controls image-to-video trajectories, and Seg2Any [253] turns open-set segmentation masks into shape- and semantic-controlled images. Generator controllability is not the controller’s decision-making scope: these methods apply a supplied condition but do not independently decide whether another operation should be invoked or whether a failed result should be repaired. The

same boundary holds for DreamFusion, Magic3D [50], and DreamGaussian [51], whose internal optimization creates 3D content under a fixed objective.

Fixed retrieval. Beyond direct generation and editing, Re-Imagen [52] supplies retrieved image-text examples through a predetermined retrieval-and-generation path. Retrieval can improve the condition presented to a generator, but it remains an L0 component when neither the retrieval result nor the generated output changes a later generation action. A controller enters L1 or above only when it decides how retrieved evidence changes the specification or subsequent route.

B. Data Construction and Training Infrastructure

Before deployment, data pipelines construct supervision and optimization procedures update model parameters. We examine this offline sequence in two stages: data construction creates or filters supervision, and fixed generator training uses that supervision or a reward to update parameters. Either stage can contain tool-using or multi-role processes, but its complexity does not determine the decision-making scope of the resulting inference system.

Data construction. ScaleEdit-12M [53] uses multiple roles to construct image-editing data, JAVEDIT [54] curates joint audio-visual editing supervision, and DataEvolver [55] evolves text-rich image data. AgentComp [56] builds compositional preference records with tool-using language models, while Gen-n-Val [57] optimizes prompts and filters synthetic instances. These systems may use agentic processes during curation, but the resulting dataset or fixed generator does not inherit their generation-level decisions.

Fixed generator training. Once supervision or feedback has been constructed, DPOK [58] optimizes diffusion parameters against image rewards, AeSlides [59] trains a slide generator with verifiable layout rewards, and FrontCoder [60] combines pre-training, supervised fine-tuning, and reinforcement learning for frontend generation. T2I-R1 [61], ReasonGen-R1 [62], and OSPO [249] likewise improve image generation through reasoning supervision or preference optimization. WorldCycle [254] and Ask, Solve, Generate [255] add self-verifiable or self-consistency rewards. All remain L0 when their deployed inference path does not choose among generation-level actions.

C. Evaluators and Benchmarks

Evaluation infrastructure measures synthesized outputs or diagnoses systems without deciding what the tested generator should do next. We distinguish evaluators and reward models, which assign judgments to synthesized outputs, from benchmarks, which organize tasks and expose capability failures. Either can support training or a later controller, but neither establishes an L1–L4 decision-making scope by itself.

Evaluators and reward models. ImageReward [63] scores text-image alignment and preference, Pick-a-Pic [64] supplies human preference data, and PIGReward [65] derives personalized evaluation dimensions. CIGEval [256] organizes conditional-image assessment through agentic evaluators. These components become part of an L3 loop only when another mechanism maps their judgment to a later revision, rerouting, or stopping action.

Benchmarks. Whereas evaluators produce scores or judgments, benchmarks define the cases over which those judgments are interpreted. T2I-CompBench [20] and GenEval [21] test compositional image requirements, while VBench [22] and EvalCrafter [23] evaluate video quality. AgenticVBench [66], DirectorBench [67], and VideoArgus [68] diagnose video-generation and editing systems. DECKBench [69], 3DCodeBench [70], and VisEditBench [71] extend evaluation to slides, procedural 3D modeling, and visualization-code editing. WeEdit [257] specifically benchmarks text-centric image editing through instruction adherence, text clarity, and background preservation. Their measurements characterize capabilities and failures, but do not select the system’s next generation action.

D. Discussion: Boundary Test for L0 Fixed Support

The final test applies to the complete deployed decision process rather than to an individual component. We first examine fixed pipelines and aggregation, then apply a counterfactual

rule that asks whether the deployed system could choose another generation action. A multi-stage pipeline remains L0 when every transition is prescribed, even if it contains retrieval, rewards, several roles, or repeated sampling.

Fixed pipelines and aggregation. Slide Translation [258] generates several layout candidates and selects one, but the selected score does not trigger another layout operation. ShareVerse [259] combines collaborative attention and spatiotemporal retrieval inside a distributed video generator without exposing a policy that chooses later generation actions. Fixed candidate ranking and internal state can improve synthesized content without creating a generation controller.

Counterfactual decision rule. To distinguish those fixed pipelines from controller-bearing systems, ask whether the deployed system can choose among materially different generation actions given the same goal and state. If no such choice exists, it remains L0. If it decides only what condition to supply to a predetermined executor, it enters L1. Selecting and invoking an operation establishes L2, using an observed outcome to change a later current-task action establishes L3, and persistent cross-task updates establish L4. This rule separates the supporting infrastructure described above from the controller levels that follow.

V. L1: CONDITIONING CONTROL

L1 Conditioning Control begins where L0 Fixed Support ends. L0 can provide a fixed generator, retriever, evaluator, or training component, but its deployed path does not decide how the task specification should change. This fixed path leaves ambiguity in the prompt, spatial arrangement, retrieved evidence, or temporal structure to the generator itself. L1 adds a pre-execution specification policy that translates the goal into an explicit condition before invoking that fixed executor. This reduces ambiguity and makes constraints inspectable, but it still commits before observing the generated result. When the system must choose and invoke a different operation after that commitment, the problem moves to L2 Execution Control.

L1 methods share the interface $c = \pi(g)$ followed by $y = G(c)$, but differ in the primary generator-facing specification they commit to an executor. We distinguish textual prompts, spatial and geometric structure, retrieved evidence, temporal and camera controls, and structured content descriptions. Reasoning, multimodal input, and role decomposition are ways to construct these specifications rather than peer categories. Figure 7 locates the commitment before execution, and Table IV summarizes the five specification types.

A. Textual Prompt Specifications

A textual specification transforms the user’s request into model-facing language while committing before any generated output is observed. Generator compatibility, user preference, and safety are competing objectives of this transformation, not separate types of controller action.

Prompt transformation and constraint preservation. Promptist [72] establishes learned prompt adaptation by optimizing model-facing text for aesthetic quality while preserving semantic intent. TIPO [73] instead uses lightweight

TABLE IV
CATEGORIES AND SUBCATEGORIES OF L1 CONDITIONING CONTROL

Category	Subcategory	Controlled decision	Representative systems
Text prompt	Prompt transformation and constraint preservation	Produce model-ready text while retaining intent and policy constraints	Promptist [72], TIPO [73], DiffChat [75], POSI [77]
Spatial/geometric	Layout and region specifications	Commit boxes, regions, or spatial relations	LMD [14], LayoutGPT [15], LLMControl [80]
	Scene and pose specifications	Commit scene structure, assets, or poses	LLM Blueprint [81], NaLA [82], DAC-Pose [83]
Retrieved evidence	Retrieval policy	Decide whether, where, and what to retrieve	World-To-Image [16], Gen-Searcher [84]
	Evidence-to-condition grounding	Associate evidence with the intended entity or attribute	Cross-modal RAG [85], RealRAG [86], MosAIG [87]
Temporal/camera	Subject, action, and temporal specifications	Commit motion, identity, and action progression	Aurora [88], TempAct [90], AgentHOI [91]
	Camera and viewpoint specifications	Commit camera paths and shot continuity	ShotVerse [97], CinemaTraj [98]
Structured content	Compositional and narrative specifications	Commit semantic, story, panel, or storyboard structure	Think-Then-Generate [100], CANVAS [102], S2ED [103]
	Document and interface specifications	Commit slide, poster, or interface structure	SlideTailor [108], ScreenCoder [109], PosterGen [110]

text presampling to expand prompts toward the language distribution of text-to-image data, and APE [74] trains compact prompt enhancers for generation and editing. Instruction-oriented systems change the objective rather than the action type. DiffChat [75] modifies prompts from explicit user instructions, ICG [76] injects personalized preference context, and POSI [77] rewrites unsafe prompts while retaining requested semantics. ThinkGen and MGIE further use multi-modal reasoning to construct model-ready generation or editing instructions [48], [78]. All belong to one branch because their controlled object is the textual condition; rewards and multimodal inputs determine how that condition is constructed.

B. Spatial and Geometric Specifications

Spatial specifications externalize geometry that a text encoder may represent weakly. We distinguish explicit layouts and regions from scene or pose descriptions, while treating reasoning and role organization as construction mechanisms.

Layout and region specifications. LMD [14] converts a complex request into object descriptions and bounding boxes that guide a frozen diffusion model. LayoutGPT [15] generalizes coordinate prediction to 2D and 3D arrangements through in-context examples, while GoT and LLMControl provide related spatial planning interfaces [79], [80]. Regional-Aware text-to-image Generation (RAG) [260] binds attributes to localized regions so that composition constraints remain explicit. These methods commit the arrangement before rendering, so

an omitted object or invalid region remains a specification error rather than an outcome-driven repair.

Scene and pose specifications. LLM Blueprint [81] represents an intended scene through a structured description. NaLA [82] combines language with native 3D asset features to construct an arrangement, and DAC-Pose [83] specializes the condition to pose-guided human composition. Although these systems use different modalities and planners, their primary commitment is geometric: the executor receives a scene, asset, or pose specification that fixes spatial relations before generation.

C. Retrieved Evidence Specifications

Retrieved evidence supplies facts or appearances absent from the fixed generator. The two decisions form a clear sequence: first choose whether and what to retrieve, then ground the selected evidence in the generator-facing condition.

Retrieval policy. World-To-Image [16] probes whether a concept failure reflects missing knowledge and retrieves definitions or reference images only when the generator lacks adequate coverage. Gen-Searcher [84] learns a multi-hop text and image search policy with textual and visual rewards. Both control the evidence acquisition step before rendering, rather than repairing a generated output afterward.

Evidence-to-condition grounding. Cross-modal RAG [85] decomposes a request into evidence roles, while RealRAG [86] aligns retrieved content with realistic synthesis. MosAIG [87]

shows why culturally specific cues must remain associated with their intended concepts. ORIG [261] combines textual facts and visual references, whereas WGen-v1 [262] studies how a single image can condition image and world construction. These methods are classified by the retrieved evidence they commit, not by whether retrieval uses text, images, or multimodal reasoning.

D. Temporal and Camera Specifications

Temporal specifications describe how content evolves, while camera specifications describe how that content is viewed. Both are committed before the predetermined video or scene executor produces an outcome.

Subject, action, and temporal specifications. Aurora [88] converts an underspecified editing request into text, references, and spatial grounding for one diffusion transformer. CogPortrait, TempAct, and AgentHOI construct portrait motion, action progression, or ordered human-object contacts [89]–[91]. VideoGen-of-Thought [92] adds shot structure and identity constraints. MovieAgent and OmniDrive use multiple roles to construct scripts, scene descriptions, and shared temporal world conditions [93], [94]. Captain Cinema, CineAGI, Sima 1.0, and InfinityStory similarly commit keyframes, character constraints, documentary structure, or shot transitions before generation [95], [96], [263], [264]. Their role count is secondary to the temporal specification that reaches the executor.

Camera and viewpoint specifications. ShotVerse [97] maps language into globally aligned camera trajectories across shots. CinemaTraj [98] grounds cinematic paths in a reconstructed 3D scene and optimizes permitted parameters against collision and occlusion constraints. LiVER and Camera Artist similarly commit illumination, viewpoint, or cinematic camera language before synthesis [99], [265]. These methods separate subject evolution from scene observation.

E. Structured Content Specifications

Some generators consume an explicit semantic or document structure rather than only text, geometry, or retrieved evidence. We separate compositional and narrative structures from schemas tied to editable documents and interfaces.

Compositional and narrative specifications. Think-Then-Generate and an offline-RL styling planner construct semantic or style decisions before image synthesis [100], [101]. CANVAS and S2ED represent story progression through storyboard or executable descriptions [102], [103], while MCCD and collaborative text-to-image generation coordinate roles to produce a compositional condition [104], [105]. MangaFlow and MM-StoryAgent add story-section, panel-layout, reference, or modality-specific conditions before generation [106], [107]. These systems are grouped by the structured content description they produce, not by reasoning or multi-role architecture.

Document and interface specifications. SlideTailor [108] commits slide content and layout preferences to a fixed realization path. ScreenCoder [109] constructs a grounded interface hierarchy and code specification, while PosterGen [110] turns a paper into content, layout, and style specifications for a poster. In each case the structure is modality-specific and established before rendered feedback could redirect execution.

F. Discussion: Commitments, Failure Modes, and Level Boundaries

Taken together, L1 methods share a central tradeoff: richer conditions reduce ambiguity and expose constraints before expensive sampling, but they also commit the system more strongly to decisions made before generation. We discuss this tradeoff through two connected questions: which failures arise at the condition interface and how portable each representation is across generators.

Failure modes. The analyzed methods reveal three distinct sources of conditioning error. Prompt optimizers can distort intent while making language more compatible with a generator. Spatial planners can omit an entity or encode an impossible arrangement. Retrieval systems can introduce irrelevant evidence that the generator follows faithfully. These failures require different diagnostics. Semantic preservation should be measured for Promptist-like rewriting, geometric validity for LayoutGPT-like planning, and evidence relevance for World-To-Image-like retrieval. A single final-image preference score cannot identify which interface failed.

Portability. Beyond diagnosing condition errors, representation choice determines how much control survives generator replacement. Natural-language prompts are broadly portable but weakly binding. Boxes, trajectories, and scene descriptions expose stronger constraints, yet they depend on a generator that understands the chosen control format. LMD and LLMControl illustrate this tradeoff for spatial control, while AgentHOI and CinemaTraj expose it for temporal and camera conditions. A useful L1 comparison should therefore report both within-generator quality and transfer across generator families.

Training changes how a specification is constructed, but not where that specification enters the workflow. Promptist, APE, Gen-Searcher, and learned motion planners optimize different condition policies, yet all must commit before observing the generated result. This limitation motivates execution control, where the controller can choose and invoke visual operations, and outcome-adaptive control, where observed results can redirect later actions.

VI. L2: EXECUTION CONTROL

L2 Execution Control changes the action interface from a specification to an executable choice $a_t \sim \pi(a_t \mid g, s_t)$. We classify systems by the primary executable object that the controller selects, invokes, or composes before observing its outcome: model or tool operations, image or structured-graphic operations, video or audiovisual operations, document or interface operations, and 3D, CAD, or world operations. Workflow construction and role coordination are implementation strategies within these five action spaces. Figure 7 contrasts L2 Execution Control with L0 Fixed Support and L1 Conditioning Control, while Table V summarizes the executable objects.

A. Model and Tool Operations

This action space is organized by whether the controller selects one operation or composes several operations into

TABLE V
CATEGORIES AND SUBCATEGORIES OF L2 EXECUTION CONTROL

Category	Subcategory	Controlled decision	Representative systems
Model/tool	Operation and model routing	Select a generator, editor, search tool, or mode	Visual ChatGPT [17], LLM-I [111], SearchGen [113]
	Multi-step workflow construction	Compose executable tool, graph, or code operations	ComfyUI-Copilot [18], ComfyUI-R1 [114], GenClaw [115]
Image/graphic	Program-level image operations	Select ordered generation or editing calls	VPGen [116], IEAP [117], ImageEdit-R1 [118]
	Element-level structured operations	Operate on layers, vectors, objects, or graphs	VisPainter [119], MiLDEdit [120], SceneCraft [121]
Video/audiovisual	Clip-level generation and editing operations	Select operations over a clip or media state	VideoAgent [123], RIVER [122]
	Shot and timeline operations	Compose scene, shot, or timeline calls	The Script is All You Need [124], DreamFactory [126], ViMax [19]
	Cross-modal audiovisual operations	Coordinate video, audio, and related media tools	LVAS-Agent [128], Mora [129], MultiMedia-Agent [130]
Document/interface	Document and interface object operations	Invoke operations over document or interface state	PresentAgent-2 [131], TVIR [132]
3D/CAD/world	Geometry and asset operations	Execute typed geometry or appearance edits	CADIR [133], Vinedresser3D [134]
	Scene, engine, and world operations	Dispatch assets, views, and engine commands	AutoUE [135], 3D Space as a Scratchpad [136], Unify-Agent [137]

an executable route. The selected tools can serve any visual modality, but the primary controlled object is the tool invocation itself.

Operation and model routing. Visual ChatGPT [17] exposes visual foundation models as callable operations and lets a language controller select a model and construct its arguments. LLM-I [111] broadens the route to search, code, generation, and editing, while Boogu-Image-0.1 [112] learns operation selection under an action budget. SearchGen [113] decides whether missing generator knowledge justifies text or image retrieval, and Mind-Brush [266] selects between generation and editing according to user intent. These systems differ in available tools, but all control which executable operation acts next.

Multi-step workflow construction. ComfyUI-Copilot [18] constructs executable workflow graphs whose nodes expose data-flow and compatibility constraints. ComfyUI-R1 [114] trains reasoning models to generate such node-based workflows, while GenClaw [115] uses code-driven canvas operations as its action language. Unlike single-step routing, these systems commit an ordered graph or program before generated outcomes can redirect it.

B. Image and Structured-Graphic Operations

Image generation and editing make the action representation visible at the output level. In addition to selecting a generator, the controller must specify what an operation changes and which structures remain editable. We therefore distinguish executable edit programs from persistent editable structure.

Program-level image operations. Programmatic editing turns a request into ordered operations before they modify pixels. VPGen [116] establishes an executable representation for generation and evaluation, while IEAP [117] specializes it to ordered diffusion operations. ImageEdit-R1 [118] learns decomposition and sequencing roles that produce an executable edit program. These systems control operation order rather than only supplying one global prompt.

Element-level structured operations. VisPainter [119] converts raster intent into vector-oriented operations, while MiLDEdit [120] exposes layer-level actions over individual design elements. SceneCraft [121] uses a graph whose nodes dispatch generators and editors over explicit content state. Persistent editability is a consequence of operating on elements, layers, vectors, or graph nodes, rather than a separate action-space category.

C. Video and Audiovisual Operations

Video action spaces differ by the temporal and modal extent of the executable object. We separate operations over one clip, operations that organize several shots or timeline units, and operations that coordinate video with audio or other media.

Clip-level generation and editing operations. RIVER [122] converts an edit request into an ordered sequence of executable operations. VideoAgent connects video understanding to generation and editing tools over the current media state [123]. These controllers choose how a clip is transformed, but their route does not depend on inspecting the resulting clip.

Shot and timeline operations. The Script is All You Need [124] lets a director orchestrate generation calls across scenes, while StoryAgent and DreamFactory construct and execute storyboard or multi-scene workflows [125], [126]. CineAgents [127] grounds a blueprint in narrative memory before one-pass tool assembly. ViMax [19] orders screenwriting, shot planning, character styling, and generation operations, with a fixed candidate-selection stage that does not trigger repair. The shared controlled object is an executable sequence over shots or scenes.

Cross-modal audiovisual operations. LVAS-Agent [128] coordinates planning and audio-generation operations over an existing temporal structure. Mora and MultiMedia-Agent expose reusable generation and editing operations across media types [129], [130]. These systems are classified here because synchronization across media organizes the route, not because they use a multi-role architecture.

D. Document and Interface Operations

This category covers executable actions over document or interface objects. Systems that only construct a slide layout, poster schema, or interface hierarchy remain in L1, while systems whose rendered or browser outcomes trigger revision belong in L3.

Document and interface object operations. PresentAgent-2 [131] invokes research, media, and presentation operations in a selected workflow. TVIR [132] similarly coordinates retrieval and visual operations to construct an interleaved report. Their controller acts on structured content objects before observing a rendered result, which separates them from L1 specification construction and L3 browser-driven repair.

E. 3D, CAD, and World Operations

Three-dimensional action spaces expose either geometry and assets or a broader scene and engine state. Both categories require executable operations, but neither uses an observed outcome to redirect the current route.

Geometry and asset operations. CADIR [133] provides a cross-backend intermediate representation whose typed features and dependencies support localized CAD operations. Vinedresser3D [134] decomposes a text-guided 3D edit into staged geometry and appearance actions. Both organize execution around geometry or asset state before rendering.

World construction makes the L2 to L3 boundary especially visible because executable constraints can become observations. A controller is L2 when it chooses environment-building operations but follows the chosen route without inspecting their results. It becomes L3 when collision, reachability, compilation, rendering, or engine state triggers a different construction action.

Scene, engine, and world operations. AutoUE [135] allocates executable tasks inside Unreal Engine, while 3D Space as a Scratchpad [136] externalizes spatial reasoning through editable scene operations. Unify-Agent [137] combines search-grounded context with world-building actions when the available generator lacks a requested entity. The primary executable object is the scene or engine workspace rather than an individual geometric feature.

F. Discussion: Cross-Domain Execution and the Open-Loop Boundary

L2 systems show that a long plan, many roles, and executable state do not by themselves establish feedback. The level changes only when an observation returned by execution affects the next generation-level action.

Across L2, action abstraction is the main architectural choice. Flat tool interfaces, as in Visual ChatGPT, make heterogeneous models immediately accessible but leave compatibility implicit in language descriptions. Workflow graphs, as in ComfyUI-Copilot, expose data types and dependencies. Code and domain-specific languages, as in GenClaw and CADIR, provide stronger execution guarantees but require the controller to satisfy a formal syntax. Structured content models occupy a middle ground because PowerPoint objects, layers, browser components, and scene graphs are executable while remaining aligned with visual concepts. These representations can be reused across image, video, document, interface, and 3D tasks, but that cross-domain reuse is a property of the action space rather than a separate L2 category.

Role decomposition and tool routing solve different problems. A router decides which visual operation should act, whereas a multi-role workflow partitions expertise and state. SearchGen focuses on whether external evidence is necessary for the selected generator. Combining role decomposition with routing can improve specialization, but additional role communication also increases latency and coordination cost.

The open-loop boundary has practical consequences for evaluation. L2 studies should verify plan validity, tool arguments, execution success, and state preservation even when the terminal output looks plausible. A workflow can obtain a strong image from an accidental route, or it can produce a weak image despite selecting the correct capabilities. Reporting both trajectory correctness and output quality separates controller failure from generator failure. It also provides the baseline needed to demonstrate L3: the same tools and initial plan should improve when rendered feedback is allowed to redirect execution.

VII. L3: OUTCOME-ADAPTIVE CONTROL

L3 adds the outcome-dependent state transition visualized in Figure 8. After a_t produces o_{t+1} , the controller updates $s_{t+1} = F(s_t, a_t, o_{t+1})$ before selecting the next action. We classify systems by the decisive observation that triggers the next current-task action: a perceptual outcome, structured or execution state, physical or hard-constraint result, or in-episode human review. Search, iterative repair, rerouting, stopping, and state preservation describe how a controller responds to feedback and are therefore secondary strategies. Table VI summarizes the four feedback sources.

A. Perceptual Outcome Feedback

Perceptual feedback is a rendered image, video segment, multiview projection, slide, or interface screenshot whose visible discrepancy changes a later action. The controller may revise a condition, search another candidate, choose a local

TABLE VI
CATEGORIES AND SUBCATEGORIES OF L3 OUTCOME-ADAPTIVE CONTROL

Category	Subcategory	Controlled decision	Representative systems
Perceptual outcome	Image and structured-content renders	Revise from visible semantic, compositional, or layout defects	RPG [138], GenArtist [267], PaperBanana [268]
	Video renders	Revise motion, composition, or camera from generated frames	GenMAC [148], MotionAgent [149], Co-Director [154]
	3D and multiview renders	Revise geometry or placement from rendered views	SceneAssistant [163], VIGA [164], WorldClaw [167]
	Document and interface renders	Revise document or code from rendered appearance	PPTAgent [172], UI2Code ^N [42], VisRefiner [177]
Structured/execution	Program and workflow state	Revise from execution results or structured content state	ComfySearch [269], GVR-Coder [185]
	Timeline and source state	Revise later edits from retained temporal state	ReCA [195], LAVE [198], Crayotter [199]
	Scene and engine state	Revise from scene, program, or runtime observations	Authoring for Living Worlds [205], HDL [206], MUSE [207]
Physical/constraint	Simulated dynamics	Revise motion or behavior from simulation results	NEWTON [219], MoReGen [221], VideoCoCo [222]
	Geometric and world constraints	Revise from collision, stability, reachability, or rule checks	PhyScensis [223], World Craft [270], MAGIC [271]
Human review	In-episode review and approval	Revise, continue, or stop from explicit human judgment	Promptify [228], CoGen3D [229], OrchestrXR [230]

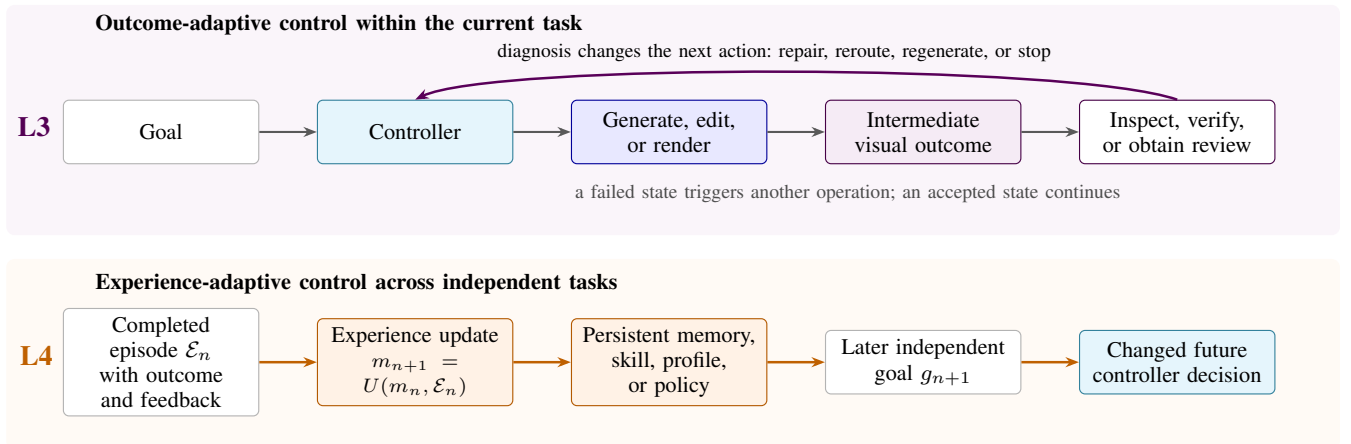


Fig. 8. The distinction between L3 Outcome-Adaptive Control and L4 Experience-Adaptive Control. L3 systems such as GenArtist [267] and PPTAgent [172] use a generated or rendered outcome to choose a later action in the same task. L4 systems such as GenEvolve [27] and COMFYCLAW [28] transform completed-task evidence into persistent procedures or workflow skills that change control on a later independent task.

edit, reroute a tool, or stop. Those responses do not create separate feedback categories.

Image and structured-content renders. RPG and SLD localize compositional failures and revise regional conditions or latent operations [24], [138]. RS-Gen and GenPilot connect rendered diagnosis to regeneration [25], [139], while Count-Loop, MetaPoint, Atelier, FiRe, VisualPrompter, narrative-product critics, and IA-T2I target counting, point placement,

style, or reference-use failures [140]–[147]. Search-based controllers allocate additional prompt, latent, or operation exploration from the current output [272]–[295].

GenArtist and T2I-Copilot map visible defects to tools or localized edits [267], [296]; related repair systems specialize regeneration, retouching, task decomposition, and stopping [297]–[315]. State-preserving editors retain accepted canvas elements while interpreting the next visual diag-

nosis [316]–[330]. Unified and latent policies implement the same render-to-action relation inside one learned controller [331]–[347]. Scientific-figure and executable-workflow systems apply rendered diagnosis to diagrams, visual explanations, or workflow outputs [268], [269], [348]–[353].

Video renders. GenMAC [148] routes a failed compositional requirement to redesign, while MotionAgent [149] changes trajectory and camera controls after inspecting generated motion. AniME, Hollywood Town, CoAgent, and MAViS apply perceptual correction at different temporal scales [150]–[153]. Automated reviewer systems similarly convert generated frames or trajectories into later generation, story, or camera actions [154]–[162], [354]–[360]. A reviewer role belongs here when it reads the rendered outcome; its architectural name does not create a distinct feedback source.

3D and multiview renders. SceneAssistant [163] revises assets and layouts from scene views, while VIGA [164] interleaves code generation, rendering, inspection, and repair. SceneConductor, Scenethesis, WorldClaw, and Kubrick correct geometry, placement, lighting, or scripts from image evidence [165]–[168]. WorldAgents, IterCAD, and ParticleGen add multiview verification, visually grounded CAD correction, or rendered particle-effect feedback [169]–[171]. Their shared limitation is observability because one attractive view cannot establish validity behind the camera or under interaction.

Document and interface renders. PPTAgent [172] uses intermediate slide renders to guide incremental editing, while PaperX, APEX, and DeepPresenter revise slide or poster content from visual diagnosis [173]–[175]. UI2Code^N, DesignCoder, VisRefiner, CITL, GameUIAgent, and AceCoder map browser screenshots to later code changes [42], [176]–[180]. These systems are classified by rendered appearance even when the repair itself operates on structured objects.

B. Structured and Execution Feedback

Structured feedback exposes program, document, timeline, scene, or runtime state. The decisive observation is an execution result or state transition rather than appearance alone.

Program and workflow state. COMIC, LayerCraft, I2E, MCTS-Report, GVR-Coder, and Any2Poster use critic state, layers, object representations, or executable validation to select a later repair [181]–[186]. Talk to Your Slides uses execution errors to trigger another structured edit [187]. OmniPresent, Auto-Slides, PreGenie, Automatic Slide Updating, PPTArena, SeaSlides, Learning to Present, ETPDesigner, and WebVIA likewise connect document or browser state to later operations [188]–[194], [361], [362]. Graph search, code repair, and localized editing are strategies for acting on this state.

Timeline and source state. ReCA [195] extracts visual, narrative, and transition state after each multi-shot unit. LA-SEV and Mind-of-Director use compiled or explicit temporal state to alter later operations [196], [197]. LAVE grounds requests in source clips and timeline operations, while Crayotter optimizes traceable long-horizon edit trajectories [198]–[200]. T2VTree, DIRECT, GLANCE, EditDuet, CutClaw, and AutoMV similarly revise source selection, ordering, synchronization, or transitions from evolving edit state [201]–[204], [363], [364].

Scene and engine state. Authoring for Living Worlds [205] checks each event against accumulated simulator state, HDSL [206] verifies and repairs a hierarchical scene program, and MUSE [207] updates requirement and preservation state after each edit. SimWorlds and MANSION use runtime or queried scene state to advance, retry, or provision another object [208], [209]. Agentic 3D Scene Generation and Plan-Craft retain spatial contracts [210], [211], while Articraft, SceneCode, FilmAgent, CutsScene Agent, StateFlow, Global-Local Monte Carlo Tree Search, and Lumera feed tests, outcomes, scripts, or world state to the controller [212]–[218].

C. Physical and Constraint Feedback

Physical feedback is distinguished from general execution state because a simulation, geometric relation, reachability test, or hard rule supplies the accept or repair condition. A renderer or engine may implement the check, but the violated constraint must change the next action.

Simulated dynamics. NEWTON [219] combines scientific computation with verifier-triggered replanning for dynamics-aware video. Environment feedback exposes collision, occlusion, or rule failures during generation [220]. MoReGen, VideoCoCo, PhysCodeBench, and GS-Agent close related loops around simulated motion, executable physical behavior, or physics-engine state [221], [222], [365], [366]. These methods differ from perceptual correction because plausible appearance does not establish correct dynamics.

Geometric and world constraints. PhyScensis uses physics-solver feedback, and SAGE gates simulation-ready scenes with physical-stability checks [223], [224]. Agentic Designer and SceneSmith correct collision, support, and stability failures during layout construction [225], [226]. iARCS adapts scene generation from constraint rewards [227], while World Craft and MAGIC use collision, connectivity, navigability, or transition checks to repair world structure [270], [271].

D. Human Review Feedback

Human review is a separate feedback source only when explicit judgment inside the episode changes a later generation action. A terminal preference study remains evaluation.

In-episode review and approval. Promptify [228] converts preferences over generated images into changes to the current prompt. CoGen3D lets a user revise or approve a concept image before image-to-3D generation, while OrchestrXR supports review and patching across study design, scene construction, and interaction logic [229], [230]. From Idea to Co-Creation [231] also lets a human supervisor redirect critic-guided revision. The criterion is causal intervention by a person, not the presence of a role named reviewer.

E. Discussion: Diagnosis, Preservation, and Stopping

L3 improves tasks by spending computation conditionally, but every additional action creates another failure opportunity. This subsection compares the properties that cut across prompt revision, tool-based editing, video control, and executable visual outputs: how diagnosis is connected to action, how

accepted state is preserved, which feedback sources expose different failures, and where within-task adaptation ends.

Diagnosis-action coupling. The strongest distinction within L3 is the coupling between diagnosis and action. Prompt refinement systems translate a visual defect back into language. Tool-based editors can instead choose a localized operation, while executable-content systems map the defect to code, XML, or scene state. The latter two provide a clearer path for credit assignment because the affected object remains identifiable. They also require more precise critics. A mistaken global prompt may degrade the whole sample, but an incorrect code edit can silently break a previously valid component.

Preservation and stopping. Once a diagnosis has selected an action, preservation and stopping become as important as correction. Agent Banana records accepted image content, ReCA maintains visual and narrative state across video segments, and browser-based systems test whether earlier interface requirements regress. These mechanisms address the same problem at different scales: a successful local repair is not progress if it destroys more valuable state elsewhere. Studies should report non-target change and regression after every accepted action, not only the quality of the final output.

Feedback sources. Whether that action is reliable depends in turn on which feedback source produced the diagnosis. Multimodal critics can detect semantic mismatch and some aesthetic defects, executable validators can expose compilation or structural errors, and simulators can test geometry or physics. No one source covers all three. NEWTON combines scientific computation with visual verification because appearance alone cannot establish physical correctness. PPTAgent and UI2Code^N combine structured execution with rendered evidence because valid files can still look wrong. A robust L3 controller should retain these signals separately and select repairs according to the violated requirement rather than collapse them prematurely into one scalar score.

Episode boundary. Finally, even reliable within-task feedback does not provide cross-task adaptation. GenPilot can remember failed actions during one request, and a long-video system can preserve state for hundreds of steps, yet neither history helps a later independent goal if it is discarded when the task ends. The next section examines systems that retain and reuse such experience.

VIII. L4: EXPERIENCE-ADAPTIVE CONTROL

L4 carries an update across the episode boundary. After episode $\mathcal{E}_n$, retained state changes as $m_{n+1} = U(m_n, \mathcal{E}_n)$ and influences later goals. We classify systems by the carrier in which experience persists: capability and tool profiles, episodic or user memory, reusable procedures and skills, executable workflows and harnesses, or policy and model updates. Image, video, document, interface, and 3D applications are examples within these carrier types rather than peer categories. Figure 8 separates this persistent update from an L3 repair loop, and Table VII summarizes the five carriers.

A. Capability and Tool Profiles

Capability profiles retain empirical evidence about which generator, editor, or tool should serve a later request. Their

persistent carrier is a routing record rather than a prior episode, executable procedure, or parameter update.

Empirical capability records. DiffusionAgent [232] stores expert descriptions that guide later diffusion-model selection. OctoT2I [26] makes those profiles empirical through a propose, solve, evaluate, and learn loop that updates future routing from measured quality and efficiency. PerfGuard [233] updates tool preferences from ranked execution evidence, while GenRouter [234] changes later workflow routing from accumulated outcomes. These profiles should be evaluated for calibration when tools, prices, or request distributions change.

B. Episodic and User Memory

Episodic memory retrieves records of prior requests, outcomes, or user preferences as context for a later independent task. Unlike a capability profile, the stored unit describes experience with a goal or user rather than a general tool estimate.

Retrieved episodes and preferences. MemoGen [235] stores relation-level successes and failures and retrieves relevant episodes for a new request. BrandFusion [236] collects user feedback from completed branded videos and reuses positive or negative integration strategies. UniVA [237] retains trajectory and user-preference memory for later video workflows, while MemSlides [238] stores hierarchical slide-generation experience for later decks. Action Agent [239] links retained navigation experience with subsequent video-generation decisions. These systems qualify only when retrieved experience changes control on a later task.

C. Reusable Procedures and Skills

Procedural memory abstracts an action pattern from completed trajectories. The retained unit is model-readable operational knowledge that must be interpreted before execution.

Abstracted action procedures. GenEvolve [27] distills successful and failed image-generation trajectories into procedures for new open-ended goals. EvoDiagram and ManimAgent accumulate reusable diagram or visual-explanation skills [240], [241], while SEAR [242] retains plans for later degradations. SceneCraft [367] stores Blender programs and spatial procedures, and SimWorld Studio [371] adds reusable engine tools and skills after verifier-guided world construction. These systems reuse an abstracted strategy rather than replay one completed output.

D. Executable Workflows and Harnesses

Executable persistence stores a graph, program, middleware component, or control harness that can organize a later workflow. This carrier can be validated and versioned because its execution semantics exceed those of a declarative skill.

Executable workflow revisions. COMFYCLAW [28] evolves typed ComfyUI graphs and promotes a workflow only after staged verification. FigAgent [243] evolves reusable drawing middleware, while AutoDesign [244] updates the harness that coordinates later design trajectories.

TABLE VII
CATEGORIES AND SUBCATEGORIES OF L4 EXPERIENCE-ADAPTIVE CONTROL

Category	Subcategory	Controlled decision	Representative systems
Capability and tool profiles	Empirical capability records	Update future model or tool routing	OctoT2I [26], DiffusionAgent [232], PerfGuard [233], GenRouter [234]
Episodic and user memory	Retrieved episodes and preferences	Condition later control on prior outcomes or users	MemoGen [235], BrandFusion [236], UniVA [237], MemSlides [238]
Procedures and skills	Abstracted action procedures	Reuse a distilled strategy on a later task	GenEvolve [27], EvoDiagram [240], SceneCraft [367]
Workflows and harnesses	Executable workflow revisions	Reuse a verified graph, program, or harness	COMFYCLAW [28], VideoWeaver [368], AutoDesign [244]
Policy and model updates	Persistent behavioral updates	Change future control through learned parameters or policy	SIDiffAgent [245], JarvisEvo [246], SPIRAL [247]
	Recursive self-improvement	Change the improvement mechanism through accepted self-updates	Gödel Agent [369], Darwin Gödel Machine [370]

VideoWeaver [368] evaluates and revises long-video workflow skills before merging them across task categories. AVA-Encoder [372] evolves a program for representing film-level state across later video tasks. In each case the retained executable object, rather than the application modality, defines the category.

E. Policy and Model Updates

The final carrier writes completed experience into a learned policy or model so later control changes without retrieving a separate memory item or workflow.

Persistent behavioral updates. SIDiffAgent [245] changes later generation behavior from completed diffusion trajectories, while JarvisEvo [246] co-evolves editing and evaluation components. SPIRAL [247] uses critic trajectories to improve behavior on later action-conditioned video goals. CLARE [248] updates its clarification policy from completed 3D interactions, and SymbOmni [373] incorporates accumulated symbolic concepts into later model behavior. Parameter and policy updates can generalize beyond explicit retrieval, but they are harder to inspect and roll back.

Recursive self-improvement. A stronger L4 case is recursive self-improvement (RSI), in which a retained update changes not only task behavior but also the mechanism that proposes or evaluates later updates. Gödel Agent makes agent logic editable under a high-level objective, whereas the Darwin Gödel Machine iteratively modifies agent code and accepts variants through empirical benchmarks [369], [370]. These systems address general-purpose agents rather than visual generation, so they are architectural precedents rather than members of the visual-generation corpus. A visual-generation controller would qualify as RSI only when an accepted change to its controller or improvement policy measurably improves its ability to generate and validate still later changes on independent visual tasks. Repeated prompt revision, self-reflection,

or training within a fixed improvement loop does not meet this condition because the improvement mechanism itself remains unchanged. RSI is therefore not an additional level in our hierarchy. It is the most self-referential form of L4 because its causal reach still crosses episode boundaries.

F. Discussion: Transfer, Failure, and Rollback

Experience can preserve mistakes as easily as useful behavior. L4 evaluation must use chronological held-out tasks and report forward transfer, negative transfer, stale-experience sensitivity, provenance, and rollback. A self-evolution label without cross-task behavioral evidence is insufficient.

RSI imposes a stricter evaluation requirement. Later generations must become better at producing validated updates, rather than only improving task performance under one fixed update procedure. Evaluation should therefore report task-level gains separately from gains in the improvement process and should use external validation to detect self-confirming regressions.

The five carriers differ in reversibility. A profile entry or episode can be deleted, a procedure can be revised, and an executable workflow can be versioned. A harmful parameter update may instead require a checkpoint or a new training run. Comparing these carriers requires measuring both later-task benefit and the cost of detecting and undoing a bad update.

IX. TRAINING AND REINFORCEMENT LEARNING FOR GENERATION CONTROLLERS

The capability taxonomy describes what a controller can change at inference time, whereas training determines how these capabilities are learned. This distinction matters because neither supervised fine-tuning nor reinforcement learning inherently determines the controller level. A fixed generator optimized with an image reward may remain in L0 Fixed Support, while a supervised router can make L2 Execution Control

decisions and an untrained repair loop can make L3 Outcome-Adaptive Control decisions. We organize controller learning into four causal stages. Trajectory data record decisions and their consequences, supervised fine-tuning initializes behavior from demonstrations, feedback provides credit signals, and policy optimization improves the resulting policy.

A. Data and Trajectory Supervision

Generation controllers act through sequences rather than isolated condition-output pairs. Thus, their training data must connect a request and controller state to an action, the resulting observation, resource use, and termination. We distinguish nominal action trajectories, failure and repair trajectories, and records that preserve long-horizon or cross-task context. This subsection concerns record content and construction, not how model parameters are updated from it.

Nominal action trajectories. A nominal controller trajectory records the ordered states, decisions, actions, and observations produced during successful generation. Useful datasets preserve plans, tool calls, intermediate results, and termination rather than retaining only final synthesized outputs. Visual ChatGPT [17] and T2I-Copilot [296] make tool and role decisions observable, while GenArtist [267] adds decomposition and verification. Temporal alignment is essential because training must recover which observation followed each action rather than merely imitate the final appearance distribution.

Failure and repair trajectories. Successful demonstrations provide weak evidence about when to revise, stop, or abandon a route. Failure and repair data retain the state before an error, the responsible action, the observed consequence, and a feasible alternative. Image-POSER [314] separates routing, editing, and stopping decisions so that poor operation selection is not confused with poor execution. Crayotter [199] applies the same principle to long video-editing sequences, where a missing event may originate in retrieval, ranking, or timeline assembly. Matched counterfactuals and rejected search branches provide clearer credit than preference pairs in which several operations change simultaneously.

Long-horizon and cross-task records. Beyond local success and failure pairs, temporal and persistent controllers require data that preserve state across longer horizons and task boundaries. SPIRAL [247] aligns plans and critic feedback with action-conditioned video segments, while VideoWeaver [368] records complete workflow experiences for subsequent skill revision. GenEvolve [27] connects completed tool orchestration with reusable procedures, and OctoT2I [26] accumulates evidence about generator capabilities and costs. Such records should also preserve provenance, tool versions, timestamps, and rollback information. Without this context, a route learned under an earlier generator or evaluator may be reused after the assumptions underlying that route have become invalid.

B. Supervised Fine-Tuning

Supervised fine-tuning (SFT) updates a generator or controller to imitate labeled outputs and action trajectories. Unlike data construction, SFT specifies a parameter-learning step.

Unlike reward-based optimization, it learns from demonstrated targets rather than assigning scalar credit to sampled alternatives. We distinguish SFT of a generator or condition policy from SFT of an executable controller trajectory.

Generator and condition SFT. Supervised targets can initialize behavior within a predetermined generation interface. Promptist [72] learns model-facing prompt rewrites from manually engineered examples before reward optimization. ReasonGen-R1 [62] fine-tunes an autoregressive image generator on explicit reasoning traces before GRPO, while FrontCoder [60] uses supervised fine-tuning as one stage of frontend generation training. These methods learn useful conditions or generator behavior, but SFT alone does not broaden the deployed action space.

Controller trajectory SFT. Whereas generator and condition SFT targets a predetermined interface, demonstrations with interleaved reasoning, operations, and observations can initialize an executable controller. GenAgent [277] learns tool invocation and reflection from multimodal trajectories before agentic reinforcement learning. ToolArtist [333] converts teacher trajectories with search and image generation into a unified multimodal format, then uses them to train reasoning, tool use, and native generation within one policy. SFT therefore provides a stable starting policy for multi-step control, while later reward optimization determines which sampled trajectories should be preferred.

C. Reward Models and Feedback

Trajectory data specify what occurred, but they do not determine which decisions deserve credit. We separate terminal outcome and preference rewards from process and verifiable rewards. Content-specific checks instantiate these two reward locations rather than forming a third peer category.

Terminal outcome and preference rewards. Terminal rewards judge the completed output and are most useful when intermediate decisions are difficult to label. ImageReward [63] and Pick-a-Pic [64] learn broad human preferences over generated images, while PIGReward [65] adapts evaluation dimensions to an individual user. Such rewards capture holistic appeal and intent better than one mechanical metric, but they cannot identify which tool call or revision caused the improvement. They should therefore supervise terminal success while process feedback handles credit within the trajectory.

Process and verifiable rewards. Because terminal rewards cannot identify which decision caused success, process feedback evaluates whether intermediate actions are valid and useful. It can test tool execution, constraint satisfaction, diagnosis-edit consistency, preservation of accepted content, and whether another action is worth its cost. GUV [374] supplies localized multimodal judgments, while GenEval [21] and T2I-CompBench [20] expose entity and relation failures that can be converted into targeted signals. AlphaGRPO [375] decomposes complex requests into verifiable components for denser reinforcement learning, although independent component success does not guarantee a coherent result. A practical

objective therefore combines goal and process evidence with action cost,

$$r_t = \alpha r_t^{\text{goal}} + \beta r_t^{\text{process}} - \lambda c(a_t), \quad (4)$$

where r_t^{goal} measures progress toward the requested output, r_t^{process} evaluates the validity and usefulness of the intermediate decision, and $c(a_t)$ is the resource cost of action a_t . The nonnegative coefficients α and β weight goal-level and process-level evidence, while λ controls the penalty on computation, tool use, or interaction cost. The components should be reported separately so that an apparent gain cannot be attributed only to the chosen weighting.

The evidence used for process verification depends on the content type. Image editing must protect accepted regions [297], [316], and video requires action-completion and temporal-state checks [148], [247]. Slides and interfaces add compilation, editability, rendering, and interaction [42], [193], while 3D scenes require geometry and simulation evidence that may be hidden from one view [163]. Across modalities, soft appearance rewards should operate only among synthesized outputs that satisfy task-specific hard constraints.

D. Reinforcement Learning and Policy Optimization

Once trajectories and feedback are defined, optimization determines which part of the generation process changes. The state can contain the request, current output, action history, verifier outputs, persistent experience, and remaining budget. The action can modify a generator, construct a condition, select a tool, revise an observed output, update retained state, or terminate. A cost-aware controller can be written as

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right], \quad (5)$$

where θ denotes the policy parameters, π_θ is the controller policy, and τ is a trajectory induced by that policy and the task environment. The horizon T is the final decision step, t indexes decisions within the trajectory, $\gamma \in [0, 1]$ is the discount factor, and r_t is the cost-aware reward defined in (4). The meaning of this objective depends on which policy is optimized and when interaction is available.

The following discussion answers three different questions and does not present four mutually exclusive algorithm families. The first two branches identify the optimization target, the third identifies when interaction is available, and the fourth identifies whether the update survives the current task.

Optimization target: generators and conditions. The narrowest target changes visual content generation within a prescribed interface. DPOK [58] fine-tunes diffusion parameters against image rewards, while AeSlides [59] and FrontCoder [60] use verifiable feedback for slide and interface generation. Condition policies instead treat language or structure as the action. Promptist [72] and DiffChat [75] optimize model-facing instructions against semantic and preference signals, and PASTA [376] adapts this decision across several user turns. These methods can improve visual quality without learning a broader workflow because the executor and invocation pattern remain fixed.

Optimization target: workflow controllers. Broader policies choose among generation operations and can condition later decisions on visual state. Policy Optimized T2I Pipeline Design [377] learns combinations of generators and processing blocks, while Image-POSER [314] and GenAgent [277] add routing, editing, reflection, and stopping. SearchGen [113] jointly recalibrates retrieval and generation. Image CoT [331], UniGen [378], and LLM-I [111] place several controller functions in one model or policy, with further realizations spanning latent, browser, and interface actions [42], [177], [274], [346]. Architecture does not determine the level; the learned policy must govern a consequential generation action.

Interaction regime. After specifying what the policy controls, learning differs by when new interaction is available. Offline methods train from fixed data and inherit its coverage and tool boundaries. Promptist [72] combines supervised initialization with reward optimization, while TIPO [73] learns low-cost prompt expansion from text distributions. Online reinforcement learning can discover routes through controlled interaction, as NEWTON [219] demonstrates inside a video generation and verification loop. Test-time methods adapt only to the current output: MILR [274] optimizes multimodal representations, and Generation Navigator [276] chooses among stopping, refinement, and regeneration. One policy can combine several regimes, so they are reported as training conditions rather than exclusive method categories.

Persistence horizon. Some updates end with the current request, while others alter later independent-task control. OctoT2I [26] updates a capability profile for future routing. GenEvolve [27] distills completed experience into procedures, COMFYCLAW [28] promotes verified workflows, and SIDiffAgent [245] retains successful and failed diffusion experience. VideoWeaver [368], VISTA [157], and SPIRAL [247] extend persistent adaptation to video behavior. These carriers differ in reversibility, but all require measurable change on a later independent task. Held-out transfer, negative transfer, stale experience, and rollback are evaluated in Section X.

The four stages expose different failure points. Incomplete trajectories hide the action that caused an outcome, supervised fine-tuning can imitate narrow or inconsistent demonstrations, weak rewards assign credit to the wrong behavior, and unconstrained optimization exploits whichever signal is easiest to increase. Reporting all four stages is therefore necessary to distinguish an improved controller from a stronger generator, a larger search budget, or a policy that overfits its evaluator.

X. EVALUATION AND BENCHMARKING

Training specifies how a controller changes, but evaluation must determine whether better decisions caused the improvement. A stronger generator, a larger sampling budget, or an evaluator shared with the controller can otherwise create the appearance of agentic progress. Our second major contribution is a *level-conditioned evaluation framework that isolates the causal value of broader controller decision-making scope by matching generators, tools, budgets, and evaluators across L1–L4*. L0 establishes the fixed-executor baseline. Each later level then changes one class of decisions and compares it with

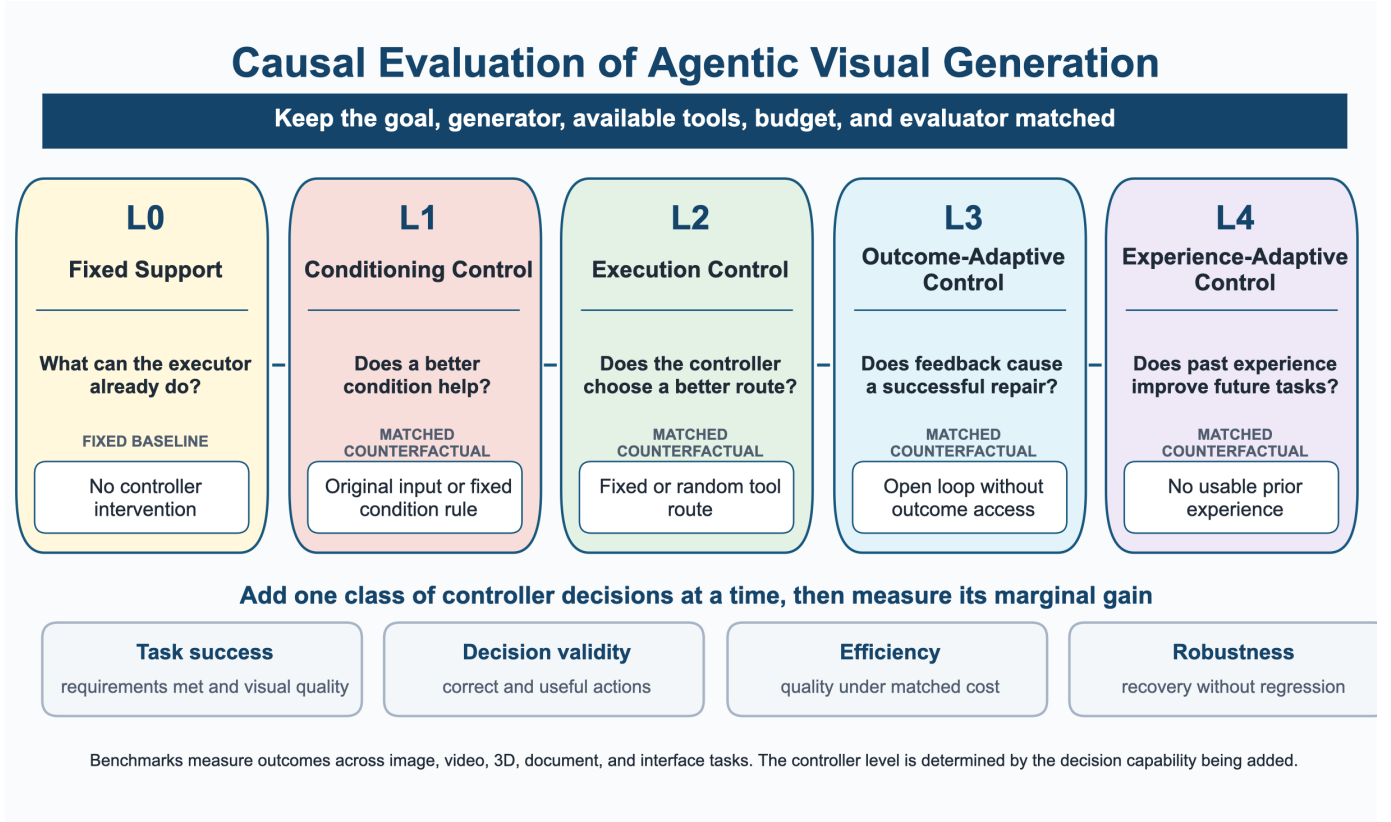


Fig. 9. Causal evaluation of agentic visual generation. The generator, available tools, budget, and evaluator remain matched while one class of controller decisions is added at each level. The comparison attributes marginal improvement to specification construction in L1 Conditioning Control, operation selection in L2 Execution Control, outcome-dependent repair in L3 Outcome-Adaptive Control, or cross-task experience reuse in L4 Experience-Adaptive Control. Content-specific benchmarks provide measurement signals but do not determine the controller level.

a matched counterfactual that removes that capability. Figure 9 presents this logic, and Table VIII specifies the corresponding controls and evidence.

A. L0 Fixed Support: What Can the Fixed Executor Already Do?

L0 Fixed Support supplies the baseline against which every claim of controller benefit must be measured. We first test synthesized-content validity and task completion, then quality and human preference, and finally how these signals should be reported without hiding hard failures through aggregation. The executor, input condition, seeds, sampling parameters, and candidate budget remain fixed throughout this baseline.

Synthesized-content validity and task completion. Evaluation should first test requirements that cannot be compensated for by visual appeal. These include requested entities and relations, exact text or counts, preservation of content during editing, file validity, executability, and action completion. The concrete checks depend on the synthesized content, but they serve the same role: an image that omits an object, a video that misses an action, a scene that violates physical state,

and an interface that fails to run are unsuccessful even when selected views appear convincing. Such requirements should be reported as separate pass rates rather than folded into a quality average.

Quality and human preference. Among synthesized outputs that satisfy their hard requirements, automatic evaluators can measure fidelity, alignment, perceptual quality, and consistency. ImageReward [63] and Pick-a-Pic [64] add learned evidence of overall human preference, while direct human evaluation remains necessary for creative intent and usability. Studies should disclose the evaluator population, instructions, comparison count, randomization, agreement, and confidence intervals. Preference and absolute quality scores remain complementary to requirement-specific checks because an attractive output can still violate the task.

Reporting and aggregation. A useful L0 report presents a vector containing hard-constraint success, synthesized-content quality, human preference, executability, and resource cost. Aggregation into one number should be avoided unless the weighting is disclosed, and hard failures should never be averaged away by a high aesthetic score. The resulting frozen report becomes the common output baseline for the L1–L4

TABLE VIII
LEVEL-CONDITIONED EVALUATION PROTOCOL

Level-specific question	Matched factors	Counterfactual	Evidence of added value
L0 Fixed Support: What can the fixed executor already do?	Input condition, random seeds, sampling parameters, candidate budget	Fixed executor without controller intervention	Requirement success, synthesized-content quality, human preference, cost
L1 Conditioning Control: Does specification control improve the same executor?	Generator, seeds, sampling parameters, candidate budget	Original input or fixed specification rule	Intent preservation, specification validity, generator compliance
L2 Execution Control: Does the controller select a better route?	Available models and tools, action budget, evaluator	Fixed, random, frequency-based, or oracle route	Routing regret, invalid actions, success and quality under matched cost
L3 Outcome-Adaptive Control: Does outcome feedback cause a successful repair?	Initial plan, tools, generator, evaluator, total budget	Open-loop execution without intermediate outcome access	Diagnosis precision, repair success, preservation, calibrated stopping
L4 Experience-Adaptive Control: Does completed experience improve future tasks?	Future-task distribution, tools, generator, budget	Removed, shuffled, stale, or irrelevant experience	Forward transfer, negative transfer, forgetting, retrieval and rollback success
Cross-level: What is the marginal value of added scope?	Generator, evaluator, goal set, calls and monetary cost	Add one decision class at a time from L1 to L4	Quality–cost frontier and marginal gain at each transition

interventions below.

B. Controls Shared by L1–L4 Evaluations

The L0 baseline establishes whether the final output succeeds. Controller evaluation additionally asks whether the trajectory is valid, efficient, robust, and causally responsible for that success. DiffusionAgent [232], GenArtist [267], OctoT2I [26], and GenEvolve [27] expose different parts of this trajectory evidence. The four dimensions below are shared reporting requirements rather than another taxonomy, and each supports the level-specific counterfactuals that follow.

Decision validity. Decision validity covers constraint coverage, dependency consistency, and correspondence between planned and executed states. It also includes invalid-call rate, argument correctness, evidence relevance, redundant actions, and whether a diagnosis predicts a repair that improves the targeted region. For persistent controllers, this dimension tests whether memory is relevant rather than stale or unrelated.

Efficiency and trajectory utility. Efficiency records model calls, generated candidates, editing operations, tokens, latency, accelerator time, and monetary cost. Let $\tau = (s_0, a_0, o_1, \dots, s_T)$ denote a trajectory, $Q(y_T)$ terminal quality, and $C(\tau)$ total resource cost. Comparisons should report the Pareto frontier of Q and C and success under fixed budgets. This prevents best-of-many sampling from being compared directly with a single-call baseline without accounting for additional search.

Robustness and recovery. This dimension measures whether the controller detects a known failure and returns to a valid route under the same remaining budget. The injected failure should target the state available at the evaluated level. L1 can receive an invalid layout or irrelevant reference, L2 an unavailable tool, L3 a corrupted intermediate outcome or critic

error, and L4 stale or unrelated experience. Reports should identify the injection point, the expected affected decision, recovery success, and any regression in previously satisfied requirements. Human controllability can be measured through edit effort, intervention count, undo success, and persistence of accepted modifications.

Faithful attribution. Attribution should be evaluated through interventions rather than the fluency of a rationale. Planned entities should correspond to rendered regions, reported actions should match tool calls, and a diagnosis should predict which repair succeeds. Counterfactual tests can remove a cited reference, alter one planned constraint, or replace a retrieved skill and check whether the claimed part of controller behavior changes.

C. L1 Conditioning Control: Does the Specification Improve a Fixed Generator?

L1 evaluation isolates the value of the specification while preventing execution differences from entering the comparison. The controller should be credited only when its prompt, layout, reference, or other condition improves the same pre-determined executor under the same sampling budget.

The generator, random seeds, sampling parameters, and candidate budget should be held fixed while the condition policy is varied. Promptist [72] should therefore be compared with the unmodified request and with supervised prompt rewriting under identical sampling conditions. LayoutGPT [15] requires an additional comparison against layouts produced without in-context spatial reasoning. Reports should separate intent preservation, specification validity, generator compliance, and terminal preference. If a rewritten prompt improves aesthetics but deletes a requested object, the condition policy has traded away goal coverage rather than solved the request. Cross-

generator evaluation is also necessary because a specification optimized for one generator may not transfer to another.

D. L2 Execution Control: Does the Controller Select a Better Route?

L2 evaluation holds the available operations fixed and varies only who decides which operation is invoked. The relevant counterfactual is not a weaker tool set. It is a fixed, random, frequency-based, or oracle route over the same generators, editors, search tools, and renderers.

Visual ChatGPT [17] gains capabilities from the models it can invoke, so a fair ablation compares its controller with a fixed routing rule over the same tool set. Mind-Brush [266] should be compared with a system that retains the same search, reasoning, generation, and editing components but fixes generation-versus-editing mode in advance. The evaluation should record routing regret, invalid calls, redundant calls, parameter errors, execution failures, cost, and the fraction of goals for which the selected route outperforms the best single-operation baseline. An oracle router supplies the attainable upper bound. Random and frequency-based routers reveal how much improvement comes from the decision policy rather than raw tool strength.

E. L3 Outcome-Adaptive Control: Does Feedback Cause a Successful Repair?

L3 evaluation must establish the full outcome-to-action link. We first compare matched open and closed loops, then audit selection bias that can mimic successful repair. A higher final score after several iterations is insufficient because repeated sampling, best-of-many selection, or an evaluator shared with the controller can produce the same pattern without a correct diagnosis or repair.

Matched open and closed loops. L3 requires paired open-loop and closed-loop executions. The open-loop variant receives the same initial goal and budget but cannot observe intermediate renders. The closed-loop variant may diagnose and revise, while the generated candidates and evaluator scores are logged at every step. For SCOPE [379], one can remove persistent commitments or replace conditional skill selection with a fixed sequence to determine whether structured state is responsible for successful repair. For NEWTON [219], freezing the initial physical specification tests whether verifier-triggered replanning corrects dynamics beyond the first plan. The main metric should not be the quality of the selected final output alone. Studies should report diagnosis precision, repair success conditioned on a correct diagnosis, regression on previously satisfied constraints, and calibrated stopping.

Selection bias. Selection bias is especially severe in iterative systems. If a method generates eight candidates and reports the best one, it should not be compared with a one-sample baseline. The evaluator that selects a candidate may also be the metric used for publication, producing circular gains. A stronger protocol separates the controller critic, the stopping critic, and the held-out evaluator. Human judgments or independent task verifiers should audit cases in which the internal

score rises but a hard requirement becomes false. Reward-model agreement should also be measured before and after optimization because a policy can actively search for blind spots that were absent in static evaluation data. SLD [24] is an informative test case because its value depends on whether feedback localizes a compositional failure rather than merely encouraging another sample. A complete L3 report should also measure non-target preservation, improvement per accepted revision, and recovery after an injected critic or tool failure.

F. L4 Experience-Adaptive Control: Does Experience Improve Future Tasks?

L4 evaluation crosses the episode boundary and must therefore preserve temporal order. The protocol should separate an experience-acquisition phase from a future-task evaluation phase. Future goals must remain disjoint from stored trajectories while still permitting transfer of a learned capability, preference, or procedure. Every retained item should carry provenance and a timestamp, and each later decision should identify which memory item, capability profile, or reusable skill influenced it. This design prevents duplicated tasks or leaked future information from being mistaken for adaptation.

OctoT2I [26] can be evaluated through routing regret before and after capability-profile updates. GenEvolve [27] requires held-out goals that can benefit from learned procedures without reproducing the stored trajectories. The decisive counterfactuals remove experience, shuffle it across unrelated tasks, replace it with stale experience, or inject an irrelevant precedent. Reports should distinguish forward transfer from negative transfer and forgetting. They should also measure retrieval accuracy, stale-memory recovery, rollback success, and the cost of maintaining the experience store.

G. Cross-Level Matched Evaluation

The four experiments above isolate one class of controller decisions at a time. Cross-level comparison preserves the budget, adds decision-making scope incrementally, and ties each gain to a logged intervention.

Budget preservation. Ablations must preserve the action budget. Removing reflection often makes a system cheaper, while removing memory can increase search. Comparing variants at unrestricted compute would conflate architecture with expenditure. Each variant should therefore be evaluated under both matched-call and matched-cost settings. The first reveals decision efficiency under equal opportunities; the second reflects practical resource use when tools have different prices and latencies. Quality-cost curves should include confidence intervals over goals and stochastic runs, because a controller may improve average quality by spending disproportionately on a small subset of difficult cases.

Incremental decision-making scope. Cross-level comparisons should test one added class of decisions at a time. Starting from an L1 condition policy, an experiment can add L2 routing, then L3 outcome feedback, and finally L4 experience reuse while preserving the same generators and evaluators. The marginal gain at each transition estimates the value of the newly enabled decision. This design is stronger than

TABLE IX
MODALITY-SPECIFIC EVALUATORS WITHIN THE LEVEL-CONDITIONED PROTOCOL

Output type	Benchmarks / evaluators	Signals	Protocol use
Image	T2I-CompBench [20], GenEval [21], Draw ALL Your Imagine [380], MME-Unify [381], AtelierEval [382], AgentGen-Bench [113]	Entities, counts, relations, alignment	Match generator for L1; tools, feedback, or experience for L2–L4
Video	VBench [22], EvalCrafter [23], AIGVE-MACS [383], UniVA-Bench [237], ActVideoGen-Bench [247], CineBench [127]	Appearance, motion, temporal state, actions	Match clip budget; separate quality from repair
3D/world	SceneCraft [367], Agentic 3D Scene Generation [210], SPIRAL [247]	Program validity, geometry, physics, transitions	Freeze engine, assets, views, simulator
Slides	PPTBench [384], PresentBench [385], PPTArena [192], PPT-Eval [386], DynaSlide [191], TSBench [187]	Edits, fidelity, design, editability	Freeze deck, object model, operations, render budget
Interfaces	Design2Code [41], FronTalk [180], Vision2Web [387]	Fidelity, runnable code, interaction, regression	Freeze browser, code environment, tests, revisions

comparing unrelated named systems whose tools, backbones, and budgets differ. It also exposes the distinct error modes introduced at each transition, such as critic amplification during feedback or stale constraints during experience reuse. The taxonomy is therefore an evaluation scaffold that attributes gains and failures to the newly enabled decision.

Logged interventions. Finally, causal claims should be tied to logged interventions. If a controller states that an object is missing, the evaluation can force the corresponding local repair and compare it with an unrelated edit. If a retrieved reference is claimed to guide style, removing or replacing it should change the relevant visual attributes. If a stored skill is credited for a later success, executing the same goal without that skill should reduce success under the same budget. Such counterfactual tests convert plans, critiques, and memories from plausible narratives into falsifiable components of the generation process.

H. Task Environments and Modality-Specific Evaluators

Level-conditioned ablations determine what must be compared, while task benchmarks supply goals and modality-specific evaluators. We first map modalities to suitable evaluators, then introduce controlled perturbations that test recovery. A benchmark does not determine the controller level by itself. The same task can test an L1 specification policy, an L2 router, an L3 repair loop, or an L4 skill library when paired with the appropriate counterfactual.

Modality-specific evaluators. Table IX summarizes how existing benchmarks instantiate the output side of this protocol. Their scores remain supporting evidence. For example, a higher GenEval score does not reveal whether the gain came from an L1 specification, L2 routing, or L3 repair unless the corresponding control variables are matched.

Controlled perturbations. Benchmark suites should instantiate the shared robustness protocol with modality-specific state changes. An image benchmark may alter a region constraint, while a video benchmark may corrupt a temporal

reference. A document benchmark can introduce an invalid file state, and a 3D benchmark can violate geometry or reachability. Each perturbation should expose the same underlying causal test: whether the observation changes the relevant controller decision and restores the violated requirement under a matched budget. The environment should freeze tool versions and retain generated outputs so that recovery cannot be attributed to an unreported executor change.

Together, these protocols separate synthesized content success from controller attribution. Content-specific metrics determine whether the output satisfies the task. Matched level counterfactuals determine whether the newly enabled decision caused the gain. Cost and robustness measurements determine whether that gain survives practical constraints. Modality-specific benchmarks supply the observations needed for these tests, but the intervention and its causal reach determine the evaluated controller level.

XI. CHALLENGES AND FUTURE DIRECTIONS

The preceding protocols do more than rank systems. They reveal what is missing at each increase in causal reach. We therefore organize open problems by the transition they block rather than by a disconnected list of mechanisms.

A. L1 to L2: From Declarative to Executable Control

The first transition requires a specification policy to assume responsibility for an operation. After establishing this execution problem, we examine two consequences: safety and provenance requirements, and the need to intervene selectively under a resource budget. DiffusionAgent [232] addresses compatible pipeline construction, while GenArtist [267] exposes coordination challenges across generators and editors.

Safety and provenance. Executable control also expands the safety and provenance surface. Retrieval, model routing, 3D asset reuse, and code execution can introduce licenses, private content, or policy violations before any final output is visible. World-To-Image [16] illustrates how

retrieved world knowledge enters a generated image, and SceneCraft [367] composes external assets through executable programs. Controllers should therefore attach machine-readable provenance to references, models, licenses, parameters, and edits. Permission-aware routing should reject an invalid operation before execution rather than rely only on filtering the export.

Budgeted intervention. The practical objective is not maximal tool use but useful intervention under a budget. A router should estimate expected quality gain, latency, monetary cost, privacy risk, and tool availability. Matched-tool evaluation is essential here because an apparent L2 gain may otherwise come entirely from access to a stronger generator.

B. L2 to L3: From Execution to Reliable Feedback Control

Once a system can act, the next transition requires observations that justify a different later action. We organize this challenge around multisource feedback, credit assignment from a diagnosis to a repair, and human feedback that must preserve accepted state. GenEval [21] makes image binding errors measurable, while VBench [22] separates several temporal properties of generated video. The difficult cases are precisely those in which a controller most needs feedback, so a fluent critique cannot be treated as reliable evidence.

Multisource feedback. Future feedback should combine localized multimodal diagnosis, specialized detectors, executable checks, simulator state, uncertainty, and human escalation. The diagnosis must also map to an actionable region, object, frame interval, or code component. Otherwise repeated generation can raise a selector score without repairing the stated defect. Independent held-out evaluators are important when one model family both creates and judges the synthesized content.

Credit assignment. Reliable observation still leaves credit assignment unresolved. A failure may originate in the prompt, retrieval, route, mask, generator, verifier, or stopping rule. GenPilot [25] demonstrates feedback-guided refinement, but a rigorous study must compare its chosen repair with counterfactual actions at the same state. Matched trajectory pairs, intervention tests, and process rewards can determine whether the diagnosis caused the improvement. These tests should preserve call and cost budgets because extra candidates are not free causal evidence.

Human feedback. Human co-creation belongs to the same transition because user feedback [388] is an outcome that should change later actions without erasing accepted work. LAVE [198] exposes editable timeline state for video, while UI2Code^N [42] repeatedly compares code with a rendered interface. Future interfaces should expose plans, masks, timelines, layouts, candidates, uncertainty, and rollback. Evaluation should measure correction effort, preservation of prior decisions, and trust calibration in addition to final preference.

C. L3 to L4: From Episodic State to Reusable Experience

Within-task state becomes L4 only when a completed trajectory changes control on a later independent task. We first consider chronological robustness to stale experience, then

use long-horizon consistency to clarify why extensive current-task state is still not L4. OctoT2I [26], GenEvolve [27], and COMFYCLAW [28] illustrate capability-profile, procedure, and workflow-skill updates across tasks.

Chronological robustness. Persistent state must be evaluated chronologically. Tool endpoints, prices, user preferences, and safety policies can change, making earlier experience stale. A robust memory policy needs timestamps, provenance, conflict detection, selective forgetting, and rollback. Held-out transfer, retrieval ablations, memory shuffling, and negative-transfer rates are stronger evidence than success on near-duplicate requests. Privacy also becomes a control property because a stored image, prompt, or preference can influence many later tasks.

Long-horizon consistency. Long-horizon consistency provides a demanding test of this distinction. VisAgent [389] preserves narrative and character state within visual stories, Agentic 3D Scene Generation [210] maintains structured scene state, and SPIRAL [247] propagates action-conditioned world state across video segments. These mechanisms may support L3 even when the state is extensive. Establishing L4 requires showing that a distilled rule or memory changes control after the original project has ended.

D. Beyond L4: Generator as Controller

The L1–L4 corpus remains predominantly controller-centric. An LLM or VLM interprets the goal and selects actions, while the visual generator executes external conditions. This division creates an interface bottleneck because each result must be rendered, encoded again, and translated into a new instruction. We examine a possible generator-as-controller regime through shared generative and control state, shorter feedback loops, persistent world state, and the control evidence that remains necessary even in a unified model.

Shared generative and control state. A generator that also controls the trajectory could act on the same state from which it constructs the synthesized content. Instead of asking a language controller to describe a defect and then converting that description back into a prompt, the policy could associate the defect with the relevant visual tokens, latent region, frame interval, scene object, or generation step. This shared state offers a finer action space for localized revision and makes it easier to preserve accepted content across iterations. GoT [79] and Image CoT [331] move in this direction by interleaving reasoning with visual generation, although their interfaces do not yet establish the complete generator-as-controller regime described here.

Shorter feedback loops. The same integration could shorten the feedback loop. Current LLM/VLM controllers repeatedly serialize visual state into language, call an external generator, and inspect another rendered result. A unified visual policy could allocate additional computation only to unresolved regions or transitions, reuse intermediate generation state, and stop without another cross-model exchange. The benefit is not architectural elegance by itself. It is the possibility of lower interaction cost, more precise repair, and clearer credit assignment between an observed failure

and the generative decision that produced it. UI2Code^N [42] demonstrates the value of making rendered feedback directly determine a later content revision, while SPIRAL [247] shows how action-conditioned generation and persistent state can be coupled over a longer horizon.

Persistent world state. Generator-side control is also attractive for persistent worlds. An external language controller usually stores a symbolic summary of what happened, while the generator separately models appearance and dynamics. If control and generation share a state representation, an action, its visual consequence, and the resulting world update can remain linked across steps. Experience from completed tasks could then improve both which action is selected and how its visual consequence is generated. Visual Generation in the New Era [13] identifies world modeling as a major stage of generator evolution. The additional requirement here is that this representation must support decisions over observation, tool use, revision, stopping, and experience reuse.

Control remains necessary. Using a single model does not automatically create agenticity. A unified model that produces synthesized content through a fixed inference path remains in L0 Fixed Support, regardless of whether it contains language and visual tokens. A generator-as-controller system must demonstrate that its evolving visual state causes different later actions and that retained experience changes later tasks. It must also address risks that are easier to audit when functions are distributed across several models, including opaque action traces, self-confirming evaluation, unsafe tool use, and harmful persistent updates. An intermediate design is therefore hybrid: the visual policy controls fine-grained generation and revision, while an external controller enforces user intent, permissions, provenance, and rollback. Evaluation should compare this design with an LLM/VLM-controlled counterpart under the same generator capacity, tools, and budget, rather than crediting a single-model implementation alone.

XII. CONCLUSION

The world-modeling direction makes the paper’s central distinction especially clear. Agentic visual generation is not simply an extension of image generation to video or 3D. It changes the unit of computation from one conditional sample to a goal-directed trajectory. Controllers ground intent, construct plans, coordinate generators and tools, create persistent visual states, judge intermediate outcomes, and adapt future actions through feedback, memory, or learning.

This work organizes agentic visual generation by the maximum causal reach of the decisions a controller can make, while a finer category and subcategory taxonomy distinguishes the technical routes taken at each level. This organization separates capability from modality, tool count, multi-role topology, training procedure, and output quality. The structured corpus shows that recent growth is concentrated in within-trajectory feedback, while persistent cross-task experience remains comparatively rare. Progress therefore depends on executable action interfaces, reliable visual verification, causal credit assignment, transparent resource accounting, selective long-term memory, editable co-creation state, and auditable

provenance. Moving control into the generator could remove the lossy round trip in which visual state is repeatedly translated into language and back into generator conditions. It could also support finer revisions, lower interaction cost, and more direct credit assignment by allowing the policy to act on the state that produced the synthesized output. A potential next regime is generator-as-controller, in which generation, observation, tool selection, trajectory revision, and experience reuse are coordinated by a unified visual policy.

REFERENCES

- [1] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, “Hierarchical text-conditional image generation with CLIP latents,” *arXiv preprint arXiv:2204.06125*, 2022.
- [2] C. Saharia *et al.*, “Photorealistic text-to-image diffusion models with deep language understanding,” in *Advances in Neural Information Processing Systems*, 2022, vol. 35, , pp. 36479–36494.
- [3] J. Yu *et al.*, “Scaling autoregressive models for content-rich text-to-image generation,” *arXiv preprint arXiv:2206.10789*, 2022.
- [4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2022, , pp. 10684–10695.
- [5] D. Podell *et al.*, “SDXL: Improving latent diffusion models for high-resolution image synthesis,” *arXiv preprint arXiv:2307.01952*, 2023.
- [6] J. Betker *et al.*, “Improving image generation with better captions,” OpenAI, Tech. Rep., 2023, [Online]. Available: <https://cdn.openai.com/papers/dall-e-3.pdf>.
- [7] J. Ho *et al.*, “Video diffusion models,” in *Advances in Neural Information Processing Systems*, 2022, vol. 35, , pp. 8633–8646.
- [8] U. Singer *et al.*, “Make-A-Video: Text-to-video generation without text-video data,” in *Proc. International Conference on Learning Representations*, 2023.
- [9] J. Ho *et al.*, “Imagen Video: High definition video generation with diffusion models,” *arXiv preprint arXiv:2210.02303*, 2022.
- [10] A. Blattmann *et al.*, “Align your latents: High-resolution video synthesis with latent diffusion models,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2023, , pp. 22563–22575.
- [11] O. Bar-Tal *et al.*, “Lumiere: A space-time diffusion model for video generation,” *arXiv preprint arXiv:2401.12945*, 2024.
- [12] Z. Xing, Q. Feng, H. Chen, Q. Dai, H. Hu, H. Xu, Z. Wu, and Y.-G. Jiang, “A survey on video diffusion models,” *ACM Computing Surveys*, 2024, vol. 57, no. 2, pp. 1–42, Nov. , doi: 10.1145/3696415.
- [13] K. Wu *et al.*, “Visual generation in the new era: An evolution from atomic mapping to agentic world modeling,” *arXiv preprint arXiv:2604.28185*, 2026.
- [14] L. Lian, B. Li, A. Yala, and T. Darrell, “LLM-grounded diffusion: Enhancing prompt understanding of text-to-image diffusion models with large language models,” *Transactions on Machine Learning Research*, 2024.
- [15] W. Feng, W. Zhu, T.-J. Fu, V. Jampani, A. Akula, X. He, S. Basu, X. E. Wang, and W. Y. Wang, “LayoutGPT: Compositional visual planning and generation with large language models,” in *Advances in Neural Information Processing Systems*, 2023, vol. 36, , pp. 18225–18250.
- [16] M. H. Son, J. Oh, S. B. Mun, J. Roh, and S. Choi, “World-to-Image: Grounding text-to-image generation with agent-driven world knowledge,” *arXiv preprint arXiv:2510.04201*, 2025.
- [17] C. Wu, S. Yin, W. Qi, X. Wang, Z. Tang, and N. Duan, “Visual ChatGPT: Talking, drawing and editing with visual foundation models,” *arXiv preprint arXiv:2303.04671*, 2023.
- [18] Z. Xu *et al.*, “ComfyUI-Copilot: An Intelligent Assistant for Automated Workflow Development,” *arXiv preprint arXiv:2506.05010*, 2025.
- [19] L. Huang *et al.*, “ViMax: Agentic Video Generation,” *arXiv preprint arXiv:2606.07649*, 2026.
- [20] K. Huang, K. Sun, E. Xie, Z. Li, and X. Liu, “T2I-CompBench: A comprehensive benchmark for open-world compositional text-to-image generation,” in *Advances in Neural Information Processing Systems*, 2023, vol. 36.
- [21] D. Ghosh, H. Hajishirzi, and L. Schmidt, “GenEval: An object-focused framework for evaluating text-to-image alignment,” *arXiv preprint arXiv:2310.11513*, 2023.

- [22] Z. Huang *et al.*, “VBench: Comprehensive benchmark suite for video generative models,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2024, , pp. 21807–21818.
- [23] Y. Liu *et al.*, “EvalCrafter: Benchmarking and evaluating large video generation models,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2024, , pp. 22139–22149.
- [24] T.-H. Wu, L. Lian, J. E. Gonzalez, B. Li, and T. Darrell, “Self-correcting LLM-controlled diffusion models,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2024, , pp. 6327–6336.
- [25] W. Ye *et al.*, “GenPilot: A multi-agent system for test-time prompt optimization in image generation,” *arXiv preprint arXiv:2510.07217*, 2025.
- [26] X. Jiang, B. Chen, G. Li, Y. Duan, R. Wang, and J. Zhang, “OctoT2I: A self-evolving agentic text-to-image router,” *arXiv preprint arXiv:2606.01803*, 2026.
- [27] S. Chen *et al.*, “GenEvolve: Self-evolving image generation agents via tool-orchestrated visual experience distillation,” *arXiv preprint arXiv:2605.21605*, 2026.
- [28] Z. Li *et al.*, “COMFYCLAW: Self-Evolving Skill Harnesses for Image Generation Workflows,” *arXiv preprint arXiv:2607.01709*, 2026.
- [29] S. Tu, Q. Dai, Z. Wu, Z.-Q. Cheng, H. Hu, and Y.-G. Jiang, “Implicit temporal modeling with learnable alignment for video recognition,” in *ICCV*, 2023.
- [30] S. Tu, T. Guan, and L. Kuang, “Multiple biological granularities network for person re-identification,” in *ICMR*, 2022.
- [31] S. Tu, Q. Dai, Z.-Q. Cheng, H. Hu, X. Han, Z. Wu, and Y.-G. Jiang, “Motioneditor: Editing video motion via content-aware diffusion,” in *CVPR*, 2024.
- [32] S. Tu, Q. Dai, Z. Zhang, S. Xie, Z.-Q. Cheng, C. Luo, X. Han, Z. Wu, and Y.-G. Jiang, “Motionfollower: Editing video motion via lightweight score-guided diffusion,” *arXiv preprint arXiv:2405.20325*, 2024.
- [33] Z. Yang, S. Tu, L. Zhang, Q. Dai, Y.-G. Jiang, and Z. Wu, “Arcflow: Unleashing 2-step text-to-image generation via high-precision non-linear flow distillation,” *arXiv preprint arXiv:2602.09014*, 2026.
- [34] S. Tu, Y. Pan, Y. Huang, X. Han, Z. Xing, Q. Dai, K. Qiu, C. Luo, and Z. Wu, “Flashportrait: 6x faster infinite portrait animation with adaptive latent prediction,” in *CVPR*, 2026.
- [35] S. Tu, Q. Tian, Z. Yang, Y. Wu, X. Han, W. Kong, J. Xiong, J.-W. Zhang, Z. Zhong, L. Bo *et al.*, “Baton: Explicit semantic blueprints for joint video-audio generation,” *arXiv preprint arXiv:2605.25195*, 2026.
- [36] S. Tu, Y. Pan, Y. Huang, X. Han, Z. Xing, Q. Dai, C. Luo, Z. Wu, and Y.-G. Jiang, “Stableavatar: Infinite-length audio-driven avatar video generation,” *arXiv preprint arXiv:2508.08248*, 2025.
- [37] S. Tu, Z. Xing, X. Han, Z.-Q. Cheng, Q. Dai, C. Luo, and Z. Wu, “Stableanimator: High-quality identity-preserving human image animation,” in *CVPR*, 2025.
- [38] S. Tu, Z. Xing, X. Han, Z.-Q. Cheng, Q. Dai, C. Luo, Z. Wu, and Y.-G. Jiang, “Stableanimator++: Overcoming pose misalignment and face distortion for human image animation,” *arXiv preprint arXiv:2507.15064*, 2025.
- [39] J. Leng, S. Tu, H. Cao, S. Xie, D. Dong, Z. Wu, and Y.-G. Jiang, “Preference score distillation: Leveraging 2d rewards to align text-to-3d generation with human preference,” *arXiv preprint arXiv:2603.01594*, 2026.
- [40] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall, “DreamFusion: Text-to-3D using 2D diffusion,” in *Proc. International Conference on Learning Representations*, 2023.
- [41] C. Si *et al.*, “Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering,” in *Proc. NAACL*, 2025.
- [42] Z. Yang *et al.*, “UI2Code^N: UI-to-Code Generation as Interactive Visual Optimization,” in *Proc. ICML*, 2026.
- [43] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” in *Proc. IEEE International Conference on Computer Vision*, 2023, , pp. 3836–3847.
- [44] C. Mou *et al.*, “T2I-Adapter: Learning adapters to dig out more controllable ability for text-to-image diffusion models,” in *Proc. AAAI Conference on Artificial Intelligence*, 2024, vol. 38, no. 5, , pp. 4296–4304.
- [45] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, “DreamBooth: Fine tuning text-to-image diffusion models for subject-driven generation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2023, , pp. 22500–22510.
- [46] H. Ye, J. Zhang, S. Liu, X. Han, and W. Yang, “IP-Adapter: Text compatible image prompt adapter for text-to-image diffusion models,” *arXiv preprint arXiv:2308.06721*, 2023.
- [47] T. Brooks, A. Holynski, and A. A. Efros, “InstructPix2Pix: Learning to follow image editing instructions,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2023, , pp. 18392–18402.
- [48] T.-J. Fu *et al.*, “Guiding Instruction-based Image Editing via Multimodal Large Language Models,” *arXiv preprint arXiv:2309.17102*, 2023.
- [49] Y. Huang *et al.*, “SmartEdit: Exploring Complex Instruction-based Image Editing with Multimodal Large Language Models,” *arXiv preprint arXiv:2312.06739*, 2023.
- [50] C.-H. Lin *et al.*, “Magic3D: High-resolution text-to-3D content creation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2023, , pp. 300–309.
- [51] J. Tang, J. Ren, H. Zhou, Z. Liu, and G. Zeng, “DreamGaussian: Generative Gaussian splatting for efficient 3D content creation,” in *Proc. International Conference on Learning Representations*, 2024.
- [52] W. Chen, H. Hu, C. Saharia, and W. W. Cohen, “Re-Imagen: Retrieval-augmented text-to-image generator,” in *Proc. International Conference on Learning Representations*, 2023.
- [53] G. Chen *et al.*, “ScaleEdit-12M: Scaling Open-Source Image Editing Data Generation via Multi-Agent Framework,” *arXiv preprint arXiv:2603.20644*, 2026.
- [54] Y. Chen *et al.*, “JAVEDIT: Joint Audio-Visual Instruction-Guided Video Editing with Agentic Data Curation,” *arXiv preprint arXiv:2606.03168*, 2026.
- [55] S. Yan *et al.*, “DataEvolver: Self-Evolving Multi-Agent Data Construction for Text-Rich Image Generation,” *arXiv preprint arXiv:2606.31537*, 2026.
- [56] A. Zarei *et al.*, “AgentComp: From Agentic Reasoning to Compositional Mastery in Text-to-Image Models,” *arXiv preprint arXiv:2512.09081*, 2025.
- [57] J.-E. Huang *et al.*, “Gen-n-Val: Agentic Image Data Generation and Validation,” *arXiv preprint arXiv:2506.04676*, 2025.
- [58] Y. Fan *et al.*, “DPK: Reinforcement learning for fine-tuning text-to-image diffusion models,” *arXiv preprint arXiv:2305.16381*, 2023.
- [59] Y. Pan *et al.*, “AeSlides: Incentivizing Aesthetic Layout in LLM-Based Slide Generation via Verifiable Rewards,” *arXiv preprint arXiv:2604.22840*, 2026.
- [60] J. Feng *et al.*, “FrontCoder: Scaling Visual Fidelity in Front-End Code Generation,” in *Findings of ACL*, 2026.
- [61] D. Jiang *et al.*, “T2I-R1: Reinforcing Image Generation with Collaborative Semantic-level and Token-level CoT,” *arXiv preprint arXiv:2505.00703*, 2025.
- [62] Y. Zhang *et al.*, “ReasonGen-R1: CoT for Autoregressive Image generation models through SFT and RL,” *arXiv preprint arXiv:2505.24875*, 2025.
- [63] J. Xu *et al.*, “ImageReward: Learning and evaluating human preferences for text-to-image generation,” in *Advances in Neural Information Processing Systems*, 2023, vol. 36.
- [64] Y. Kirstain, A. Polyak, U. Singer, S. Matiana, J. Penna, and O. Levy, “Pick-a-Pic: An open dataset of user preferences for text-to-image generation,” in *Advances in Neural Information Processing Systems*, 2023, vol. 36, , pp. 36652–36663.
- [65] J. Lee *et al.*, “Personalized Reward Modeling for Text-to-Image Generation,” *arXiv preprint arXiv:2511.19458*, 2025.
- [66] Z. Cao *et al.*, “AgenticVBench: Can AI Agents Complete Real-World Post-Production Tasks?” *arXiv preprint arXiv:2605.27705*, 2026.
- [67] J. Chen *et al.*, “DirectorBench: Diagnosing Long-Form Video Generation with Personalized Multi-Agent Evaluation,” *arXiv preprint arXiv:2605.30090*, 2026.
- [68] Z. Zeng *et al.*, “VideoArgus: Agentic Rubric-Grounded Unified Evaluation for Video Generation and Editing,” *arXiv preprint arXiv:2608.05485*, 2026.
- [69] D. Jang *et al.*, “DECKBench: Benchmarking Multi-Agent Frameworks for Academic Slide Generation and Editing,” *arXiv preprint arXiv:2602.13318*, 2026.
- [70] Y. Gao *et al.*, “3DCodeBench: Benchmarking Agentic Procedural 3D Modeling Via Code,” *arXiv preprint arXiv:2606.01057*, 2026.
- [71] M. Rahman *et al.*, “VisEditBench: Can Vision-Language Models Edit Visualization Code from Multimodal Feedback?” *arXiv preprint arXiv:2608.10408*, 2026.
- [72] Y. Hao *et al.*, “Optimizing Prompts for Text-to-Image Generation,” *arXiv preprint arXiv:2212.09611*, 2022.
- [73] S.-Y. Yeh *et al.*, “TIPO: Text to Image with Text Presampling for Prompt Optimization,” *arXiv preprint arXiv:2411.08127*, 2024.
- [74] Z. Huang *et al.*, “APE: Agentic Prompt Enhancer for Image Generation and Editing,” *arXiv preprint arXiv:2606.00204*, 2026.

- [75] J. Wang *et al.*, “DiffChat: Learning to Chat with Text-to-Image Synthesis Models for Interactive Image Creation,” *arXiv preprint arXiv:2403.04997*, 2024.
- [76] Z. Bian *et al.*, “ICG: Improving Cover Image Generation via MLLM-based Prompting and Personalized Preference Alignment,” *arXiv preprint arXiv:2605.27374*, 2026.
- [77] Z. Wu *et al.*, “Universal Prompt Optimizer for Safe Text-to-Image Generation,” *arXiv preprint arXiv:2402.10882*, 2024.
- [78] S. Jiao *et al.*, “ThinkGen: Generalized Thinking for Visual Generation,” *arXiv preprint arXiv:2512.23568*, 2025.
- [79] R. Fang *et al.*, “GoT: Unleashing reasoning capability of multimodal large language model for visual generation and editing,” *arXiv preprint arXiv:2503.10639*, 2025.
- [80] J. Wang *et al.*, “LLMControl: Grounded Control of Text-to-Image Diffusion-based Synthesis with Multimodal LLMs,” *arXiv preprint arXiv:2507.19939*, 2025.
- [81] H. Gani *et al.*, “LLM Blueprint: Enabling Text-to-Image Generation with Complex and Detailed Prompts,” *arXiv preprint arXiv:2310.10640*, 2023.
- [82] C. Wan *et al.*, “NaLA: A 3D Native LLM Layout Agent for High-quality 3D Scene Generation,” *arXiv preprint arXiv:2606.29395*, 2026.
- [83] H. Yang *et al.*, “DAC-Pose: Dual-Agent Collaborative Framework for Pose-Guided Human Generation,” *arXiv preprint arXiv:2608.04622*, 2026.
- [84] K. Feng *et al.*, “Gen-Searcher: Reinforcing Agentic Search for Image Generation,” *arXiv preprint arXiv:2603.28767*, 2026.
- [85] M. Zhu *et al.*, “Cross-modal RAG: Sub-dimensional Text-to-Image Retrieval-Augmented Generation,” *arXiv preprint arXiv:2505.21956*, 2025.
- [86] Y. Lyu *et al.*, “RealRAG: Retrieval-augmented Realistic Image Generation via Self-reflective Contrastive Learning,” *arXiv preprint arXiv:2502.00848*, 2025.
- [87] P. Bhalerao *et al.*, “When Cultures Meet: Multicultural Text-to-Image Generation,” *arXiv preprint arXiv:2502.15972*, 2025.
- [88] Y. Yu *et al.*, “Aurora: Unified Video Editing with a Tool-Using Agent,” *arXiv preprint arXiv:2605.18748*, 2026.
- [89] H. Feng *et al.*, “CogPortrait: Fine-Grained Eye-Region Control in Portrait Animation via Hierarchical Agent Planning,” *arXiv preprint arXiv:2605.28056*, 2026.
- [90] J. Wang *et al.*, “TempAct: Advancing Temporal Plausibility in Autoregressive Video Generation via Planner-Executor RL,” *arXiv preprint arXiv:2606.28016*, 2026.
- [91] Z. Huang *et al.*, “AgentHOI: Multi-Agent Reasoning for Human-Object-Interaction Video Generation via Implicit Representation Alignment,” *arXiv preprint arXiv:2607.22241*, 2026.
- [92] M. Zheng *et al.*, “VideoGen-of-Thought: Step-by-step generating multi-shot video with minimal manual intervention,” *arXiv preprint arXiv:2412.02259*, 2024.
- [93] W. Wu *et al.*, “Automated Movie Generation via Multi-Agent CoT Planning,” *arXiv preprint arXiv:2503.07314*, 2025.
- [94] Z. Meng *et al.*, “OmniDrive: An LLM-Choreographed Multi-Agent World Model with Unified Latent Co-Compression for Multi-View Driving Video Generation,” *arXiv preprint arXiv:2606.17536*, 2026.
- [95] J. Xiao *et al.*, “Captain Cinema: Towards Short Movie Generation,” *arXiv preprint arXiv:2507.18634*, 2025.
- [96] T. Xie *et al.*, “CineAGI: Character-Consistent Movie Creation through LLM-Orchestrated Multi-Modal Generation and Cross-Scene Integration,” *arXiv preprint arXiv:2604.23579*, 2026.
- [97] S. Yang *et al.*, “ShotVerse: Advancing Cinematic Camera Control for Text-Driven Multi-Shot Video Creation,” *arXiv preprint arXiv:2603.11421*, 2026.
- [98] Q. Li *et al.*, “CinemaTraj: Composing Atomic Camera Trajectories for 3D Scenes with LLM Agents,” *arXiv preprint arXiv:2607.26910*, 2026.
- [99] Z. Cai *et al.*, “Lighting-grounded Video Generation with Renderer-based Agent Reasoning,” *arXiv preprint arXiv:2604.07966*, 2026.
- [100] S. Kou *et al.*, “Think-Then-Generate: Reasoning-Aware Text-to-Image Diffusion with LLM Encoders,” *arXiv preprint arXiv:2601.10332*, 2026.
- [101] S. Mukherjee *et al.*, “Agentic Planning with Reasoning for Image Styling via Offline RL,” *arXiv preprint arXiv:2603.07148*, 2026.
- [102] I. Mondal *et al.*, “CANVAS: Continuity-Aware Narratives via Visual Agentic Storyboarding,” *arXiv preprint arXiv:2604.13452*, 2026.
- [103] S. Yin *et al.*, “S2ED: From Story to Executable Descriptions for Consistency-Aware Story Illustration,” *arXiv preprint arXiv:2605.22448*, 2026.
- [104] M. Li *et al.*, “MCCD: Multi-Agent Collaboration-based Compositional Diffusion for Complex Text-to-Image Generation,” *arXiv preprint arXiv:2505.02648*, 2025.
- [105] J. Shi *et al.*, “Collaborative Text-to-Image Generation via Multi-Agent Reinforcement Learning and Semantic Fusion,” *arXiv preprint arXiv:2510.10633*, 2025.
- [106] M. Wang *et al.*, “MangaFlow: An End-to-End Agentic Framework for Controllable Story to Manga Generation,” *arXiv preprint arXiv:2605.28173*, 2026.
- [107] X. Xu *et al.*, “MM-StoryAgent: Immersive Narrated Storybook Video Generation with a Multi-Agent Paradigm across Text, Image and Audio,” *arXiv preprint arXiv:2503.05242*, 2025.
- [108] W. Zeng *et al.*, “SlideTailor: Personalized Presentation Slide Generation for Scientific Papers,” in *Proc. AAAI*, 2026.
- [109] Y. Jiang *et al.*, “ScreenCoder: Advancing Visual-to-Code Generation for Front-End Automation via Modular Multimodal Agents,” *arXiv preprint arXiv:2507.22827*, 2025.
- [110] Z. Zhang *et al.*, “PosterGen: Aesthetic-Aware Multi-Modal Paper-to-Poster Generation via Multi-Agent LLMs,” *arXiv preprint arXiv:2508.17188*, 2025.
- [111] Z. Guo *et al.*, “LLM-I: LLMs are Naturally Interleaved Multimodal Creators,” *arXiv preprint arXiv:2509.13642*, 2025.
- [112] G. Chen *et al.*, “Boogu-Image-0.1: Boosting Open Agentic Multimodal Generation via Understanding under a Minimal Budget,” *arXiv preprint arXiv:2607.13125*, 2026.
- [113] H. Wang *et al.*, “Search Beyond What Can Be Taught: Evolving the Knowledge Boundary in Agentic Visual Generation,” *arXiv preprint arXiv:2607.05382*, 2026.
- [114] Z. Xu *et al.*, “ComfyUI-R1: Exploring Reasoning Models for Workflow Generation,” *arXiv preprint arXiv:2506.09790*, 2025.
- [115] J. Ye *et al.*, “GenClaw: Code-Driven Agentic Image Generation,” *arXiv preprint arXiv:2605.30248*, 2026.
- [116] J. Cho *et al.*, “Visual Programming for Text-to-Image Generation and Evaluation,” *arXiv preprint arXiv:2305.15328*, 2023.
- [117] Y. Hu *et al.*, “Image Editing As Programs with Diffusion Models,” *arXiv preprint arXiv:2506.04158*, 2025.
- [118] Y. Zhao *et al.*, “ImageEdit-R1: Boosting Multi-Agent Image Editing via Reinforcement Learning,” *arXiv preprint arXiv:2603.08059*, 2026.
- [119] J. Sun *et al.*, “From Pixels to Paths: A Multi-Agent Framework for Editable Scientific Illustration,” *arXiv preprint arXiv:2510.27452*, 2025.
- [120] Z. Lin *et al.*, “MiLDEdit: Reasoning-Based Multi-Layer Design Document Editing,” *arXiv preprint arXiv:2601.04589*, 2026.
- [121] D.-M. Phan *et al.*, “SceneCraft: Interactive System for Image Editing via Scene Graph,” *arXiv preprint arXiv:2606.16103*, 2026.
- [122] Y. Shen *et al.*, “Text-Driven Reasoning Video Editing via Reinforcement Learning on Digital Twin Representations,” *arXiv preprint arXiv:2511.14100*, 2025.
- [123] H. Zhou *et al.*, “VideoAgent: All-in-One Framework for Video Understanding and Editing,” *arXiv preprint arXiv:2606.23327*, 2026.
- [124] C. Mu *et al.*, “The Script is All You Need: An Agentic Framework for Long-Horizon Dialogue-to-Cinematic Video Generation,” *arXiv preprint arXiv:2601.17737*, 2026.
- [125] P. Hu *et al.*, “StoryAgent: Customized Storytelling Video Generation via Multi-Agent Collaboration,” *arXiv preprint arXiv:2411.04925*, 2024.
- [126] Z. Xie *et al.*, “DreamFactory: Pioneering Multi-Scene Long Video Generation with a Multi-Agent Framework,” *arXiv preprint arXiv:2408.11788*, 2024.
- [127] P. Zhang *et al.*, “A Benchmark and Multi-Agent System for Instruction-driven Cinematic Video Compilation,” *arXiv preprint arXiv:2604.10456*, 2026.
- [128] Y. Zhang *et al.*, “Long-Video Audio Synthesis with Multi-Agent Collaboration,” *arXiv preprint arXiv:2503.10719*, 2025.
- [129] Z. Yuan *et al.*, “Mora: Enabling Generalist Video Generation via A Multi-Agent Framework,” *arXiv preprint arXiv:2403.13248*, 2024.
- [130] D. Zhang *et al.*, “A Versatile Multimodal Agent for Multimedia Content Generation,” *arXiv preprint arXiv:2601.03250*, 2026.
- [131] W. Wu *et al.*, “PresentAgent-2: Towards Generalist Multimodal Presentation Agents,” *arXiv preprint arXiv:2605.11363*, 2026.
- [132] X. Ma *et al.*, “TVIR: Building Deep Research Agents Towards Text-Visual Interleaved Report Generation,” *arXiv preprint arXiv:2606.02320*, 2026.
- [133] Y. Liu *et al.*, “CADIR: A Cross-Backend Editable Intermediate Representation for Agentic CAD Generation,” *arXiv preprint arXiv:2608.00891*, 2026.

- [134] Y. Chi *et al.*, “Vinedresser3D: Agentic Text-guided 3D Editing,” *arXiv preprint arXiv:2602.19542*, 2026.
- [135] L. Yin *et al.*, “AutoUE: Automated Generation of 3D Games in Unreal Engine via Multi-Agent Systems,” *arXiv preprint arXiv:2603.07106*, 2026.
- [136] O. Saha *et al.*, “3D Space as a Scratchpad for Editable Text-to-Image Generation,” *arXiv preprint arXiv:2601.14602*, 2026.
- [137] S. Chen *et al.*, “Unify-Agent: A Unified Multimodal Agent for World-Grounded Image Synthesis,” *arXiv preprint arXiv:2603.29620*, 2026.
- [138] L. Yang, Z. Yu, C. Meng, M. Xu, S. Ermon, and B. Cui, “Mastering text-to-image diffusion: Recaptioning, planning, and generating with multimodal LLMs,” in *Proc. International Conference on Machine Learning*, 2024.
- [139] F. Bian *et al.*, “RS-Gen: A Multi-Stage Agentic Framework for Reasoning and Search-Augmented Image Generation,” *arXiv preprint arXiv:2606.23221*, 2026.
- [140] A. Mondal *et al.*, “CountLoop: Training-Free High-Instance Image Generation via Iterative Agent Guidance,” *arXiv preprint arXiv:2508.16644*, 2025.
- [141] D. Zhou *et al.*, “MetaPoint: Unlocking Precise Spatial Control in Agentic Visual Generation,” *arXiv preprint arXiv:2606.05031*, 2026.
- [142] K. Xing *et al.*, “Beyond Starry Night: Shortcut-Aware Control-State Planning for Artist-Grounded Text to Image Generation,” *arXiv preprint arXiv:2608.06751*, 2026.
- [143] Y. Kim *et al.*, “FiRe: Fine-grained Multimodal Reasoning for Enhanced Image Generation,” *arXiv preprint arXiv:2604.13491*, 2026.
- [144] S. Wu *et al.*, “VisualPrompter: Semantic-Aware Prompt Optimization with Visual Feedback for Text-to-Image Synthesis,” *arXiv preprint arXiv:2506.23138*, 2025.
- [145] M. Luo *et al.*, “Self-Reasoning Agentic Framework for Narrative Product Grid-Collage Generation,” *arXiv preprint arXiv:2604.16958*, 2026.
- [146] J. Wang *et al.*, “Twin Co-Adaptive Dialogue for Progressive Image Generation,” *arXiv preprint arXiv:2504.14868*, 2025.
- [147] C. Li *et al.*, “IA-T2I: Internet-Augmented Text-to-Image Generation,” *arXiv preprint arXiv:2505.15779*, 2025.
- [148] K. Huang *et al.*, “GenMAC: Compositional Text-to-Video Generation with Multi-Agent Collaboration,” *arXiv preprint arXiv:2412.04440*, 2024.
- [149] X. Liao *et al.*, “MotionAgent: Fine-grained Controllable Video Generation via Motion Field Agent,” *arXiv preprint arXiv:2502.03207*, 2025.
- [150] L. Zhang *et al.*, “AniME: Adaptive Multi-Agent Planning for Long Animation Generation,” *arXiv preprint arXiv:2508.18781*, 2025.
- [151] Z. Wei *et al.*, “Hollywood Town: Long-Video Generation via Cross-Modal Multi-Agent Orchestration,” *arXiv preprint arXiv:2510.22431*, 2025.
- [152] Q. Zeng *et al.*, “CoAgent: Collaborative Planning and Consistency Agent for Coherent Video Generation,” *arXiv preprint arXiv:2512.22536*, 2025.
- [153] Q. Wang *et al.*, “MAViS: A Multi-Agent Framework for Long-Sequence Video Storytelling,” *arXiv preprint arXiv:2508.08487*, 2025.
- [154] Y. Song *et al.*, “Co-Director: Agentic Generative Video Storytelling,” *arXiv preprint arXiv:2604.24842*, 2026.
- [155] Y. Shi *et al.*, “One Sentence, One Drama: Personalized Short-Form Drama Generation via Multi-Agent Systems,” *arXiv preprint arXiv:2605.22144*, 2026.
- [156] Y. Li *et al.*, “Anim-Director: A Large Multimodal Model Powered Agent for Controllable Animation Video Generation,” *arXiv preprint arXiv:2408.09787*, 2024.
- [157] D. X. Long *et al.*, “VISTA: A Test-Time Self-Improving Video Generation Agent,” *arXiv preprint arXiv:2510.15831*, 2025.
- [158] Y. Song *et al.*, “Soap2Soap: Long Cinematic Video Remaking via Multi-Agent Collaboration,” *arXiv preprint arXiv:2605.17423*, 2026.
- [159] B. Gao *et al.*, “BOOKAGENT: Orchestrating Safety-Aware Visual Narratives via Multi-Agent Cognitive Calibration,” *arXiv preprint arXiv:2604.16541*, 2026.
- [160] H. Yan *et al.*, “AnimeAgent: Is the Multi-Agent via Image-to-Video models a Good Disney Storytelling Artist?” *arXiv preprint arXiv:2602.20664*, 2026.
- [161] Y. Song *et al.*, “VQQA: An Agentic Approach for Video Evaluation and Quality Improvement,” *arXiv preprint arXiv:2603.12310*, 2026.
- [162] C. Yang *et al.*, “SCMAPR: Self-Correcting Multi-Agent Prompt Refinement for Complex-Scenario Text-to-Video Generation,” *arXiv preprint arXiv:2604.05489*, 2026.
- [163] J. Luo *et al.*, “SceneAssistant: A Visual Feedback Agent for Open-Vocabulary 3D Scene Generation,” *arXiv preprint arXiv:2603.12238*, 2026.
- [164] S. Yin *et al.*, “Vision-as-Inverse-Graphics Agent via Interleaved Multimodal Reasoning,” *arXiv preprint arXiv:2601.11109*, 2026.
- [165] J. Kim *et al.*, “SceneConductor: 3D Scene Generation from a Single Image with Multi-Agent Orchestration,” *arXiv preprint arXiv:2606.08402*, 2026.
- [166] L. Ling *et al.*, “Scenethesis: A language and vision agentic framework for 3D scene generation,” *arXiv preprint arXiv:2505.02836*, 2025.
- [167] C. Guo *et al.*, “WorldClaw: Agentic 3D Open-World Generation at Scale,” *arXiv preprint arXiv:2608.05248*, 2026.
- [168] L. He *et al.*, “Kubrick: Multimodal Agent Collaborations for Synthetic Video Generation,” *arXiv preprint arXiv:2408.10453*, 2024.
- [169] Z. Erkoc *et al.*, “WorldAgents: Can Foundation Image Models be Agents for 3D World Models?” *arXiv preprint arXiv:2603.19708*, 2026.
- [170] T. Hu *et al.*, “IterCAD: An Iterative Multimodal Agent for Visually-Grounded CAD Generation and Editing,” *arXiv preprint arXiv:2606.13368*, 2026.
- [171] J. Zhuge *et al.*, “ParticleGen: A Multi-Agent System for Particle Effects Generation,” *arXiv preprint arXiv:2608.00629*, 2026.
- [172] H. Zheng *et al.*, “PPTAgent: Generating and Evaluating Presentations Beyond Text-to-Slides,” in *Proc. EMNLP*, 2025.
- [173] T. Yu *et al.*, “PaperX: A Unified Framework for Multimodal Academic Presentation Generation with Scholar DAG,” *arXiv preprint arXiv:2602.03866*, 2026.
- [174] C. Shi *et al.*, “APEX: Academic Poster Editing Agentic Expert,” *arXiv preprint arXiv:2601.04794*, 2026.
- [175] H. Zheng *et al.*, “DeepPresenter: Environment-Grounded Reflection for Agentic Presentation Generation,” *arXiv preprint arXiv:2602.22839*, 2026.
- [176] Y. Chen *et al.*, “DesignCoder: Hierarchy-Aware and Self-Correcting UI Code Generation with Large Language Models,” *arXiv preprint arXiv:2506.13663*, 2025.
- [177] J. Deng *et al.*, “VisRefiner: Learning from Visual Differences for Screenshot-to-Code Generation,” *arXiv preprint arXiv:2602.05998*, 2026.
- [178] H. Sansford *et al.*, “Vision-Guided Iterative Refinement for Frontend Code Generation,” *arXiv preprint arXiv:2604.05839*, 2026.
- [179] W. Zeng *et al.*, “GameUIAgent: An LLM-Powered Framework for Automated Game UI Design with Structured Intermediate Representation,” *arXiv preprint arXiv:2603.14724*, 2026.
- [180] X. Wu *et al.*, “FronTalk: Benchmarking Front-End Development as Conversational Code Generation with Multi-Modal Feedback,” *arXiv preprint arXiv:2601.04203*, 2026.
- [181] S. Hong *et al.*, “COMIC: Agentic Sketch Comedy Generation,” *arXiv preprint arXiv:2603.11048*, 2026.
- [182] Y. Zhang *et al.*, “LayerCraft: Enhancing Text-to-Image Generation with CoT Reasoning and Layered Object Integration,” *arXiv preprint arXiv:2504.00010*, 2025.
- [183] J. Yu *et al.*, “I2E: From Image Pixels to Actionable Interactive Environments for Text-Guided Image Editing,” *arXiv preprint arXiv:2601.03741*, 2026.
- [184] T. Lin *et al.*, “Monte Carlo Tree Search for Table-to-Multimodal Report Generation,” *arXiv preprint arXiv:2608.04071*, 2026.
- [185] Y. Xu *et al.*, “GVR-Coder: A Visual-Feedback Framework for Structured SVG Generation in Complex Document and Meeting Scenarios,” *arXiv preprint arXiv:2607.28073*, 2026.
- [186] A. Vinaykumar *et al.*, “Any2Poster: Any-Source Poster Generation Across Modalities and Domains,” *arXiv preprint arXiv:2606.02915*, 2026.
- [187] K. Jung *et al.*, “Talk to Your Slides: High-Efficiency Slide Editing via Language-Driven Structured Data Manipulation,” in *Findings of ACL*, 2026.
- [188] Q. Ma *et al.*, “OmniPresent: Generating Coherent Presentation Suites from Scientific Papers,” *arXiv preprint arXiv:2607.02590*, 2026.
- [189] Y. Yang *et al.*, “Auto-Slides: An Interactive Multi-Agent System for Creating and Customizing Research Presentations,” *arXiv preprint arXiv:2509.11062*, 2025.
- [190] X. Xu *et al.*, “PreGenie: An Agentic Framework for High-quality Visual Presentation Generation,” *arXiv preprint arXiv:2505.21660*, 2025.
- [191] K. Zhou *et al.*, “Automatic Slide Updating with User-Defined Dynamic Templates and Natural Language Instructions,” *arXiv preprint arXiv:2604.17894*, 2026.
- [192] M. Ofengenden *et al.*, “PPTArena: A Benchmark for Agentic PowerPoint Editing,” *arXiv preprint arXiv:2512.03042*, 2025.
- [193] S. Fang *et al.*, “SeaSlides: Semantic Abstraction Layer for Agentic Slide Generation,” *arXiv preprint arXiv:2608.03298*, 2026.

- [194] K. R. A. Kumar *et al.*, “Learning to Present: Inverse Specification Rewards for Agentic Slide Generation,” *arXiv preprint arXiv:2603.16839*, 2026.
- [195] A. Liu *et al.*, “ReCA: Multi-Shot Long Video Extrapolation via Recursive Context Allocation,” *arXiv preprint arXiv:2605.26525*, 2026.
- [196] L. Yan *et al.*, “Beyond End-to-End Video Models: An LLM-Based Multi-Agent System for Educational Video Generation,” *arXiv preprint arXiv:2602.11790*, 2026.
- [197] S. Nan *et al.*, “Mind-of-Director: Multi-modal Agent-Driven Film Previsualization via Collaborative Decision-Making,” *arXiv preprint arXiv:2603.14790*, 2026.
- [198] B. Wang, Y. Li, Z. Lv, H. Xia, Y. Xu, and R. Sodhi, “LAVE: LLM-powered agent assistance and language augmentation for video editing,” *arXiv preprint arXiv:2402.10294*, 2024.
- [199] L. Yan *et al.*, “Crayotter: Learning Long-Horizon Video Editing Agents via Group-Relative Preference Backpropagation,” *arXiv preprint arXiv:2608.02694*, 2026.
- [200] —, “Crayotter: Traceable Multi-Agent Workflows for Long-Form Video Editing,” *arXiv preprint arXiv:2606.07636*, 2026.
- [201] Z. Zheng *et al.*, “T2VTree: User-Centered Visual Analytics for Agent-Assisted Thought-to-Video Authoring,” *arXiv preprint arXiv:2602.08368*, 2026.
- [202] K. Li *et al.*, “DIRECT: Video Mashup Creation via Hierarchical Multi-Agent Planning and Intent-Guided Editing,” *arXiv preprint arXiv:2604.04875*, 2026.
- [203] Z. Lin *et al.*, “GLANCE: A Global-Local Coordination Multi-Agent Framework for Music-Grounded Non-Linear Video Editing,” *arXiv preprint arXiv:2604.05076*, 2026.
- [204] M. Sandoval-Castaneda *et al.*, “EditDuet: A Multi-Agent System for Video Non-Linear Editing,” *arXiv preprint arXiv:2509.10761*, 2025.
- [205] N. Cudlenco *et al.*, “Authoring for Living Worlds: Tool-Constrained LLM Agents for Executable Multi-Actor Scenarios,” *arXiv preprint arXiv:2604.10383*, 2026.
- [206] L. Li *et al.*, “HDSL: A Hierarchical Domain-Specific Language for Structured 3D Indoor Scene Generation and Localized Editing with LLM Agents,” *arXiv preprint arXiv:2606.09738*, 2026.
- [207] R. Xu *et al.*, “MUSE: Agentic 3D Scene Authoring via Memory-Grounded Incremental Requirement Satisfaction,” *arXiv preprint arXiv:2606.14168*, 2026.
- [208] C. Liu *et al.*, “SimWorlds: A Multi-Agent System for Dynamic 3D Scene Creation,” *arXiv preprint arXiv:2607.01766*, 2026.
- [209] L. Che *et al.*, “MANSON: Multi-floor LAnguage-to-3D Scene generation for loNg-horizon tasks,” *arXiv preprint arXiv:2603.11554*, 2026.
- [210] X. Liu, Y.-W. Tai, and C.-K. Tang, “Agentic 3D scene generation with spatially contextualized VLMs,” *arXiv preprint arXiv:2505.20129*, 2025.
- [211] P. Zeng *et al.*, “PlanCraft: Sketch, Refine, and Furnish for Architect-Inspired Progressive 3D Residential Scene Generation,” *arXiv preprint arXiv:2607.23491*, 2026.
- [212] M. Zhou *et al.*, “Articraft: An Agentic System for Scalable Articulated 3D Asset Generation,” *arXiv preprint arXiv:2605.15187*, 2026.
- [213] P. Wang *et al.*, “SceneCode: Executable World Programs for Editable Indoor Scenes with Articulated Objects,” *arXiv preprint arXiv:2605.19587*, 2026.
- [214] Z. Xu *et al.*, “FilmAgent: A Multi-Agent Framework for End-to-End Film Automation in Virtual 3D Spaces,” *arXiv preprint arXiv:2501.12909*, 2025.
- [215] L. He *et al.*, “Cutscene Agent: An LLM Agent Framework for Automated 3D Cutscene Generation,” *arXiv preprint arXiv:2604.25318*, 2026.
- [216] Y. Yin *et al.*, “StateFlow: Building, Evolving, and Accessing 3D World States for Previsualization,” *arXiv preprint arXiv:2608.12314*, 2026.
- [217] M. Qi *et al.*, “Global-Local Monte Carlo Tree Search in Vision-Language Models for Text-to-3D Indoor Scene Generation,” *arXiv preprint arXiv:2606.06002*, 2026.
- [218] J. Chen *et al.*, “Engine-Native Editable 3D World Reconstruction with Objects and Lighting,” *arXiv preprint arXiv:2607.20889*, 2026.
- [219] Y. Feng *et al.*, “NEWTON: Agentic Planning for Physically Grounded Video Generation,” *arXiv preprint arXiv:2605.18396*, 2026.
- [220] A. Soni *et al.*, “VideoAgent: Self-Improving Video Generation,” *arXiv preprint arXiv:2410.10076*, 2024.
- [221] X. Bai *et al.*, “MoReGen: Multi-Agent Motion-Reasoning Engine for Code-based Text-to-Video Synthesis,” *arXiv preprint arXiv:2512.04221*, 2025.
- [222] H. Li *et al.*, “VideoCoCo: Code-as-CoT for Physically-Consistent Video Generation via an Agentic Dual-Engine System,” *arXiv preprint arXiv:2607.27380*, 2026.
- [223] Y. Wang *et al.*, “PhyScensis: Physics-Augmented LLM Agents for Complex Physical Scene Arrangement,” *arXiv preprint arXiv:2602.14968*, 2026.
- [224] H. Xia *et al.*, “SAGE: Scalable Agentic 3D Scene Generation for Embodied AI,” *arXiv preprint arXiv:2602.10116*, 2026.
- [225] Z. Yang *et al.*, “Agentic Designer: Progressive Multi-Agent Collaboration for Structure-Aware Interior Layout Generation,” *arXiv preprint arXiv:2607.20866*, 2026.
- [226] N. Pfaff *et al.*, “SceneSmith: Agentic Generation of Simulation-Ready Indoor Scenes,” *arXiv preprint arXiv:2602.09153*, 2026.
- [227] S. Adhikari *et al.*, “iARCS: Iterative Agentic RL for Controllable 3D Scene Generation,” *arXiv preprint arXiv:2608.06161*, 2026.
- [228] S. Brade *et al.*, “Promptify: Text-to-Image Generation through Interactive Prompt Exploration with Large Language Models,” *arXiv preprint arXiv:2304.09337*, 2023.
- [229] W. Jiang *et al.*, “CoGen3D: An Agentic Human-AI Co-Design Pipeline for 3D Asset Generation for Virtual Reality,” *arXiv preprint arXiv:2607.03731*, 2026.
- [230] S. Liao *et al.*, “OrchestrXR: A Multi-Agent System for Idea-to-Prototype XR Study Authoring,” *arXiv preprint arXiv:2607.01588*, 2026.
- [231] J. Gao *et al.*, “From Idea to Co-Creation: A Planner-Actor-Critic Framework for Agent Augmented 3D Modeling,” *arXiv preprint arXiv:2601.05016*, 2026.
- [232] J. Qin, J. Wu, W. Chen, and Y. Lyu, “DiffusionAgent: Navigating expert models for agentic image generation,” *arXiv preprint arXiv:2401.10061*, 2024.
- [233] Z. Chen *et al.*, “PerfGuard: A Performance-Aware Agent for Visual Content Generation,” *arXiv preprint arXiv:2601.22571*, 2026.
- [234] H. H. Chen *et al.*, “GenRouter: Unified Workflow Routing for Agentic Image Generation,” *arXiv preprint arXiv:2608.16721*, 2026.
- [235] W. Chen *et al.*, “MemoGen: Can Past Experience Improve Future Text-to-Image Generation?” *arXiv preprint arXiv:2606.03243*, 2026.
- [236] Z. Zhu *et al.*, “BrandFusion: A Multi-Agent Framework for Seamless Brand Integration in Text-to-Video Generation,” *arXiv preprint arXiv:2603.02816*, 2026.
- [237] Z. Liang *et al.*, “UniVA: Universal Video Agent towards Open-Source Next-Generation Video Generalist,” *arXiv preprint arXiv:2511.08521*, 2025.
- [238] Y. Jin *et al.*, “MemSlides: A Hierarchical Memory Driven Agent Framework for Personalized Slide Generation with Multi-turn Local Revision,” *arXiv preprint arXiv:2606.17162*, 2026.
- [239] J. Sam *et al.*, “Action Agent: Agentic Video Generation Meets Flow-Constrained Diffusion,” *arXiv preprint arXiv:2605.01477*, 2026.
- [240] T. Wang *et al.*, “EvoDiagram: Agentic Editable Diagram Creation via Design Expertise Evolution,” *arXiv preprint arXiv:2604.09568*, 2026.
- [241] W. Jiang *et al.*, “ManimAgent: Self-Evolving Multimodal Agents for Visual Education,” *arXiv preprint arXiv:2606.30296*, 2026.
- [242] S. Cui *et al.*, “Self-Evolving Agentic Image Restoration via Deliberate Planning and Intuitive Execution,” *arXiv preprint arXiv:2606.28971*, 2026.
- [243] Z. Li *et al.*, “Automatic Method Illustration Generation for AI Scientific Papers via Drawing Middleware Creation, Evolution, and Orchestration,” *arXiv preprint arXiv:2603.29590*, 2026.
- [244] Y. Luo *et al.*, “AutoDesign: Meta-Harness Optimization for Long-Horizon Agentic Design,” *arXiv preprint arXiv:2608.13560*, 2026.
- [245] S. Garg *et al.*, “SIDiffAgent: Self-Improving Diffusion Agent,” *arXiv preprint arXiv:2602.02051*, 2026.
- [246] Y. Lin *et al.*, “JarvisEvo: Towards a Self-Evolving Photo Editing Agent with Synergistic Editor-Evaluator Optimization,” *arXiv preprint arXiv:2511.23002*, 2025.
- [247] Y. Yang *et al.*, “SPIRAL: Self-evolving action-conditioned video generation via reflective planning agents,” *arXiv preprint arXiv:2603.08403*, 2026.
- [248] X. Zhu *et al.*, “Clarify Before Executing: A Self-Evolving Agent for Resolving Intent Asymmetry in 3D Tool Orchestration,” *arXiv preprint arXiv:2607.16352*, 2026.
- [249] Y. Oh *et al.*, “OSPO: Object-Centric Self-Improving Preference Optimization for Text-to-Image Generation,” *arXiv preprint arXiv:2506.02015*, 2025.
- [250] H. Zhang, D. Hong, T. Gao, Y. Wang, J. Shao, X. Wu, Z. Wu, and Y.-G. Jiang, “CreatiLayout: Siamese multimodal diffusion transformer for creative layout-to-image generation,” in *Proc. IEEE/CVF International Conference on Computer Vision*, 2025.
- [251] H. Zhang, D. Hong, M. Yang, Y. Cheng, Z. Zhang, J. Shao, X. Wu, Z. Wu, and Y.-G. Jiang, “CreatiDesign: A unified multi-conditional

- diffusion transformer for creative graphic design,” *arXiv preprint arXiv:2505.19114*, 2025.
- [252] Q. Li, Z. Xing, R. Wang, H. Zhang, Q. Dai, and Z. Wu, “Magic-Motion: Controllable video generation with dense-to-sparse trajectory guidance,” in *Proc. IEEE/CVF International Conference on Computer Vision*, 2025, , pp. 12112–12123.
- [253] D. Li, H. Zhang, S. Wang, J. Li, and Z. Wu, “Seg2Any: Open-set segmentation-mask-to-image generation with precise shape and semantic control,” in *Advances in Neural Information Processing Systems*, 2025.
- [254] B. Gu *et al.*, “WorldCycle: Self-Verifiable Reinforcement Learning for Long-Horizon Video World Models,” *arXiv preprint arXiv:2608.04964*, 2026.
- [255] R. Thawkar *et al.*, “Ask, Solve, Generate: Self-Evolving Unified Multimodal Understanding and Generation via Self-Consistency Rewards,” *arXiv preprint arXiv:2606.27376*, 2026.
- [256] J. Wang *et al.*, “A Unified Agentic Framework for Evaluating Conditional Image Generation,” *arXiv preprint arXiv:2504.07046*, 2025.
- [257] H. Zhang, J. Liu, Z. Liu, L. Niu, F. Meng, Z. Wu, and Y.-G. Jiang, “WeEdit: A dataset, benchmark and glyph-guided framework for text-centric image editing,” *arXiv preprint arXiv:2603.11593*, 2026.
- [258] F. Kajita *et al.*, “Presentation Slide Translation and Layout Error Correction by LLMs,” in *Proc. ACL Student Research Workshop*, 2026.
- [259] J. Zhu *et al.*, “ShareVerse: Multi-Agent Consistent Video Generation for Shared World Modeling,” *arXiv preprint arXiv:2603.02697*, 2026.
- [260] Z. Chen *et al.*, “Region-Aware Text-to-Image Generation via Hard Binding and Soft Refinement,” *arXiv preprint arXiv:2411.06558*, 2024.
- [261] Y. Tian *et al.*, “Open Multimodal Retrieval-Augmented Factual Image Generation,” *arXiv preprint arXiv:2510.22521*, 2025.
- [262] K. Zeng *et al.*, “One Image is All You Need: Agentic One-Shot Image Generation via Text-Based World Models for Long-Tail Spatial Perception,” *arXiv preprint arXiv:2606.20764*, 2026.
- [263] Z. Song *et al.*, “Sima 1.0: A Collaborative Multi-Agent Framework for Documentary Video Production,” *arXiv preprint arXiv:2604.07721*, 2026.
- [264] M. Elmoghany *et al.*, “InfinityStory: Unlimited Video Generation with World Consistency and Character-Aware Shot Transitions,” *arXiv preprint arXiv:2603.03646*, 2026.
- [265] H. Hu *et al.*, “Camera Artist: A Multi-Agent Framework for Cinematic Language Storytelling Video Generation,” *arXiv preprint arXiv:2604.09195*, 2026.
- [266] J. He *et al.*, “Mind-Brush: Integrating Agentic Cognitive Search and Reasoning into Image Generation,” *arXiv preprint arXiv:2602.01756*, 2026.
- [267] Z. Wang, A. Li, Z. Li, and X. Liu, “GenArtist: Multimodal LLM as an agent for unified image generation and editing,” in *Advances in Neural Information Processing Systems*, 2024, vol. 37, , pp. 128374–128395.
- [268] D. Zhu *et al.*, “PaperBanana: Automating Academic Illustration for AI Scientists,” *arXiv preprint arXiv:2601.23265*, 2026.
- [269] J. Su *et al.*, “ComfySearch: Autonomous Exploration and Reasoning for ComfyUI Workflows,” *arXiv preprint arXiv:2601.04060*, 2026.
- [270] J. Sun *et al.*, “World Craft: Agentic Framework to Create Visualizable Worlds via Text,” *arXiv preprint arXiv:2601.09150*, 2026.
- [271] T. H. Fan *et al.*, “MAGIC: Transition-Aware Generation of Navigable Multi-Scene Game Worlds with Large Language Models,” *arXiv preprint arXiv:2607.11594*, 2026.
- [272] O. Manas *et al.*, “Improving Text-to-Image Consistency via Automatic Prompt Optimization,” *arXiv preprint arXiv:2403.17804*, 2024.
- [273] S. Kim *et al.*, “Reward-Agnostic Prompt Optimization for Text-to-Image Diffusion Models,” *arXiv preprint arXiv:2506.16853*, 2025.
- [274] Y. Mi *et al.*, “MILR: Improving Multimodal Image Generation via Test-Time Latent Reasoning,” *arXiv preprint arXiv:2509.22761*, 2025.
- [275] K. Wang *et al.*, “ImAgent: A Unified Multimodal Agent Framework for Test-Time Scalable Image Generation,” *arXiv preprint arXiv:2511.11483*, 2025.
- [276] J. Liu *et al.*, “Generation Navigator: A State-Aware Agentic Framework for Image Generation,” *arXiv preprint arXiv:2605.17969*, 2026.
- [277] K. Jiang *et al.*, “GenAgent: Scaling Text-to-Image Generation via Agentic Multimodal Reasoning,” *arXiv preprint arXiv:2601.18543*, 2026.
- [278] M. A. H. Khan *et al.*, “Test-time Prompt Refinement for Text-to-Image Models,” *arXiv preprint arXiv:2507.22076*, 2025.
- [279] D. Xiang *et al.*, “PromptSculptor: Multi-Agent Based Text-to-Image Prompt Optimization,” *arXiv preprint arXiv:2509.12446*, 2025.
- [280] T. Lee *et al.*, “PointT2I: LLM-based text-to-image generation via keypoints,” *arXiv preprint arXiv:2506.01370*, 2025.
- [281] B. Yang *et al.*, “M3: High-fidelity Text-to-Image Generation via Multi-Modal, Multi-Agent and Multi-Round Visual Reasoning,” *arXiv preprint arXiv:2602.06166*, 2026.
- [282] R. Shalev-Arkushin *et al.*, “ImageRAG: Dynamic Image Retrieval for Reference-Guided Image Generation,” *arXiv preprint arXiv:2502.09411*, 2025.
- [283] J. Li *et al.*, “A Unified Multi-Agent Framework for Universal Multimodal Understanding and Generation,” *arXiv preprint arXiv:2508.10494*, 2025.
- [284] S. Li *et al.*, “MuLan: Multimodal-LLM Agent for Progressive and Interactive Multi-Object Diffusion,” *arXiv preprint arXiv:2402.12741*, 2024.
- [285] C. Li *et al.*, “coDrawAgents: A Multi-Agent Dialogue Framework for Compositional Image Generation,” *arXiv preprint arXiv:2603.12829*, 2026.
- [286] X. Wan *et al.*, “Maestro: Self-Improving Text-to-Image Generation via Agent Orchestration,” *arXiv preprint arXiv:2509.10704*, 2025.
- [287] J. Sun *et al.*, “Marmot: Object-Level Self-Correction via Multi-Agent Reasoning,” *arXiv preprint arXiv:2504.20054*, 2025.
- [288] P. Chen *et al.*, “Agentic Flow Steering and Parallel Rollout Search for Spatially Grounded Text-to-Image Generation,” *arXiv preprint arXiv:2603.18627*, 2026.
- [289] L. Jiang *et al.*, “RAISE: Requirement-Adaptive Evolutionary Refinement for Training-Free Text-to-Image Alignment,” *arXiv preprint arXiv:2603.00483*, 2026.
- [290] G. Liang *et al.*, “Render-in-the-Loop: Vector Graphics Generation via Visual Self-Feedback,” *arXiv preprint arXiv:2604.20730*, 2026.
- [291] G. Tang *et al.*, “PRISM: Prompt Refinement via Image-grounded Self-rewarding Mechanism for Text-to-Image Generation,” *arXiv preprint arXiv:2607.24353*, 2026.
- [292] J. Chen *et al.*, “PairCoder++: Pair Programming as a Universal Paradigm for Verified Code-Driven Multimodal and Structured-Content Generation,” *arXiv preprint arXiv:2607.01883*, 2026.
- [293] Z. Wang *et al.*, “VisPuzzle: Task-Aware Composite Visualization Construction,” *arXiv preprint arXiv:2608.11635*, 2026.
- [294] Z. Wang, E. Xie, A. Li, Z. Wang, X. Liu, and Z. Li, “Divide and conquer: Language models can plan and self-correct for compositional text-to-image generation,” *arXiv preprint arXiv:2401.15688*, 2024.
- [295] A.-G. Vaduva *et al.*, “Code2UML: Agentic LLMs with context engineering for scalable software visualization,” *arXiv preprint arXiv:2605.24453*, 2026.
- [296] C.-Y. Chen, M. Shi, G. Zhang, and H. Shi, “T2I-Copilot: A training-free multi-agent text-to-image system for enhanced prompt interpretation and interactive generation,” *arXiv preprint arXiv:2507.20536*, 2025.
- [297] Z. Xu *et al.*, “EditRefiner: A Human-Aligned Agentic Framework for Image Editing Refinement,” *arXiv preprint arXiv:2605.07457*, 2026.
- [298] J. Guo *et al.*, “Refinement via Regeneration: Enlarging Modification Space Boosts Image Refinement in Unified Multimodal Models,” *arXiv preprint arXiv:2604.25636*, 2026.
- [299] K. Venkatesh *et al.*, “CREA: A Collaborative Multi-Agent Framework for Creative Image Editing and Generation,” *arXiv preprint arXiv:2504.05306*, 2025.
- [300] H. Zhao *et al.*, “Crafter: A Multi-Agent Harness for Editable Scientific Figure Generation from Diverse Inputs,” *arXiv preprint arXiv:2605.30611*, 2026.
- [301] Z. Zhang *et al.*, “What to Edit Next: Visually Aligned Image-Editing Follow-Up Suggestions in Conversational Systems,” *arXiv preprint arXiv:2608.07565*, 2026.
- [302] S. Shen *et al.*, “Agentic Retoucher for Text-To-Image Generation,” *arXiv preprint arXiv:2601.02046*, 2026.
- [303] M. Yao *et al.*, “PhotoAgent: Exploratory Visual Aesthetic Planning with Large Vision Models,” *arXiv preprint arXiv:2602.22809*, 2026.
- [304] C.-H. Yeh *et al.*, “Beyond Simple Edits: X-Planner for Complex Instruction-Based Image Editing,” *arXiv preprint arXiv:2507.05259*, 2025.
- [305] Z. Liang *et al.*, “An LLM-LVLM Driven Agent for Iterative and Fine-Grained Image Editing,” *arXiv preprint arXiv:2508.17435*, 2025.
- [306] Z. Zeng *et al.*, “MIRA: Multimodal Iterative Reasoning Agent for Image Editing,” *arXiv preprint arXiv:2511.21087*, 2025.
- [307] Q. Wu *et al.*, “RetouchIQ: MLLM Agents for Instruction-Based Image Retouching with Generalist Reward,” *arXiv preprint arXiv:2602.17558*, 2026.
- [308] Z. Qiu *et al.*, “MSRAMIE: Multimodal Structured Reasoning Agent for Multi-instruction Image Editing,” *arXiv preprint arXiv:2603.16967*, 2026.
- [309] Y. Pu *et al.*, “CAMEO: A Conditional and Quality-Aware Multi-Agent Image Editing Orchestrator,” *arXiv preprint arXiv:2604.03156*, 2026.

- [310] B. Zhao *et al.*, “Making Image Editing Easier via Adaptive Task Reformulation with Agentic Executions,” *arXiv preprint arXiv:2604.15917*, 2026.
- [311] A. S. Rajan *et al.*, “From Plans to Pixels: Learning to Plan and Orchestrate for Open-Ended Image Editing,” *arXiv preprint arXiv:2605.15181*, 2026.
- [312] Z. Zhu *et al.*, “IEA: Amateur-Friendly Conversational Image Editing Agent via Three Stages of Multitask Alignment,” *arXiv preprint arXiv:2606.08016*, 2026.
- [313] J. Gao *et al.*, “Taming I2V models for Image HOI Editing: A Cognitive Benchmark and Agentic Self-Correcting Framework,” *arXiv preprint arXiv:2606.19073*, 2026.
- [314] H. Mohebbi *et al.*, “Image-POSER: Reflective RL for Multi-Expert Image Generation and Editing,” *arXiv preprint arXiv:2511.11780*, 2025.
- [315] S. Hu *et al.*, “Domain-Grounded Candidate Selection for Agentic Image Editing: A Shadow Removal Case,” *arXiv preprint arXiv:2608.06075*, 2026.
- [316] R. Ye *et al.*, “Agent Banana: High-Fidelity Image Editing with Agentic Thinking and Tooling,” *arXiv preprint arXiv:2602.09084*, 2026.
- [317] S. Ma *et al.*, “Talk2Image: A Multi-Agent System for Multi-Turn Image Generation and Editing,” *arXiv preprint arXiv:2508.06916*, 2025.
- [318] F. Shen *et al.*, “IMAGAgent: Orchestrating Multi-Turn Image Editing via Constraint-Aware Planning and Reflection,” *arXiv preprint arXiv:2603.29602*, 2026.
- [319] Z. Yan *et al.*, “GlyphBanana: Advancing Precise Text Rendering Through Agentic Workflows,” *arXiv preprint arXiv:2603.12155*, 2026.
- [320] Y. Lin *et al.*, “JarvisHub: An Open Harness for Canvas-Native Multimodal Creative Agents,” *arXiv preprint arXiv:2607.23588*, 2026.
- [321] H. Zhu *et al.*, “CanvasAgent: Enabling Complex Image Creation and Editing via Visual Tool Orchestration,” *arXiv preprint arXiv:2607.05465*, 2026.
- [322] F. Wang *et al.*, “Cognitive-structured Multimodal Agent for Multimodal Understanding, Generation, and Editing,” *arXiv preprint arXiv:2607.08497*, 2026.
- [323] Z. Guo *et al.*, “GMO-E2DIT: Grounded Multi-Operation Editing for E-Commerce Images,” *arXiv preprint arXiv:2607.00920*, 2026.
- [324] Q. Li *et al.*, “What Can I Edit? Open-Ended Strategy Discovery and the Emotion Editability Landscape,” *arXiv preprint arXiv:2607.23920*, 2026.
- [325] P. Cao *et al.*, “DrawAI: Agentic Benchmark and Workflow for Making Raster Images Editable,” *arXiv preprint arXiv:2608.00548*, 2026.
- [326] R. Aufschlager *et al.*, “Towards Context-Aware Image Anonymization with Multi-Agent Reasoning,” *arXiv preprint arXiv:2603.27817*, 2026.
- [327] T. Sivertsen *et al.*, “SAGE: Structured Agentic Graph Editing for Software Diagrams,” *arXiv preprint arXiv:2607.01102*, 2026.
- [328] S. Huang *et al.*, “SciFig: Towards Automating Editable Figure Generation for Scientific Papers,” *arXiv preprint arXiv:2601.04390*, 2026.
- [329] H. Hu *et al.*, “PosterMELD: Multi-Agent Paper-to-Poster Generation for Controllable Design Diversity with Editable Print-Ready Outputs,” *arXiv preprint arXiv:2608.02218*, 2026.
- [330] C. Shao *et al.*, “LiveFigure: Generating Editable Scientific Illustration with VLM Agents,” *arXiv preprint arXiv:2605.23527*, 2026.
- [331] Z. Guo, R. Zhang, C. Tong, Z. Zhao, P. Gao, H. Li, and P.-A. Heng, “Can we generate images with CoT? Let’s verify and reinforce image generation step by step,” *arXiv preprint arXiv:2501.13926*, 2025.
- [332] L. L. Chen *et al.*, “UniT: Unified Multimodal Chain-of-Thought Test-time Scaling,” *arXiv preprint arXiv:2602.12279*, 2026.
- [333] J. Zhao *et al.*, “ToolArtist: Tool-Using Unified Multimodal Models for Agentic Image Generation,” *arXiv preprint arXiv:2608.04436*, 2026.
- [334] Z. Zhang *et al.*, “Qwen-Image-Agent: Bridging the Context Gap in Real-World Image Generation,” *arXiv preprint arXiv:2606.26907*, 2026.
- [335] D. Wang *et al.*, “UniReason 1.0: A Unified Reasoning Framework for World Knowledge Aligned Image Generation and Editing,” *arXiv preprint arXiv:2602.02437*, 2026.
- [336] D. Jiang *et al.*, “DraCo: Draft as CoT for Text-to-Image Preview and Rare Concept Generation,” *arXiv preprint arXiv:2512.05112*, 2025.
- [337] L. Qin *et al.*, “Uni-cot: Towards Unified Chain-of-Thought Reasoning Across Text and Vision,” *arXiv preprint arXiv:2508.05606*, 2025.
- [338] Y. Lyu *et al.*, “StruVis: Enhancing Reasoning-based Text-to-Image Generation via Thinking with Structured Vision,” *arXiv preprint arXiv:2603.06032*, 2026.
- [339] J. Lai *et al.*, “VisionCreator: A Native Visual-Generation Agentic Model with Understanding, Thinking, Planning and Creation,” *arXiv preprint arXiv:2603.02681*, 2026.
- [340] Z. He *et al.*, “GEMS: Agent-Native Multimodal Generation with Memory and Skills,” *arXiv preprint arXiv:2603.28088*, 2026.
- [341] Q. Liu *et al.*, “Breaking Dual Bottlenecks: Evolving Unified Multimodal Models into Self-Adaptive Interleaved Visual Reasoners,” *arXiv preprint arXiv:2605.14709*, 2026.
- [342] S. Ren *et al.*, “Large Language Models are Universal Reasoners for Visual Generation,” *arXiv preprint arXiv:2605.04040*, 2026.
- [343] W. Huang *et al.*, “Interleaving Reasoning for Better Text-to-Image Generation,” *arXiv preprint arXiv:2509.06945*, 2025.
- [344] D. Zheng *et al.*, “InterleaveThinker: Reinforcing Agentic Interleaved Generation,” *arXiv preprint arXiv:2606.13679*, 2026.
- [345] L. Zhang *et al.*, “Think in Strokes, Not Pixels: Process-Driven Image Generation via Interleaved Reasoning,” *arXiv preprint arXiv:2604.04746*, 2026.
- [346] F. Zhai *et al.*, “Latent Action Control for Reasoning-Guided Unified Image Generation,” *arXiv preprint arXiv:2605.16961*, 2026.
- [347] Y. Wang *et al.*, “Towards Enhanced Image Generation Via Multimodal Chain of Thought in Unified Generative Models,” *arXiv preprint arXiv:2503.01298*, 2025.
- [348] M. Zhu *et al.*, “AutoFigure: Generating and Refining Publication-Ready Scientific Illustrations,” *arXiv preprint arXiv:2602.03828*, 2026.
- [349] Z. Wen *et al.*, “Feynman: Knowledge-Infused Diagramming Agent for Scalable Visual Designs,” *arXiv preprint arXiv:2603.12597*, 2026.
- [350] R. Malik *et al.*, “Exploring Agentic Workflows for Generating High Quality Math Visual Aids,” *arXiv preprint arXiv:2607.09839*, 2026.
- [351] O. Huang *et al.*, “ComfyGPT: A Self-Optimizing Multi-Agent System for Comprehensive ComfyUI Workflow Generation,” *arXiv preprint arXiv:2503.17671*, 2025.
- [352] L. Guo *et al.*, “ComfyMind: Toward General-Purpose Generation via Tree-Based Planning and Reactive Feedback,” *arXiv preprint arXiv:2505.17908*, 2025.
- [353] Z. Li *et al.*, “Knowledge-Centric Agents for Workflow Generation in ComfyUI,” *arXiv preprint arXiv:2607.15845*, 2026.
- [354] D. Das *et al.*, “Genflow Ad Studio: A Compound AI Architecture for Brand-Aligned, Self-Correcting Video Generation,” *arXiv preprint arXiv:2605.16748*, 2026.
- [355] D. Savitski *et al.*, “Bridging Creative Intent and Visual Quality: Creator-Driven Recurrent Video Generation with Agentic Feedback Loops,” *arXiv preprint arXiv:2606.18591*, 2026.
- [356] X. Yin *et al.*, “Closed-Loop Triplet Synergistic Generation for Long-Form Video,” *arXiv preprint arXiv:2606.16184*, 2026.
- [357] S. Jun *et al.*, “Temporal Backtracking Search for Test-time Generative Video Reasoning,” *arXiv preprint arXiv:2606.13861*, 2026.
- [358] J. Zuo *et al.*, “FilmWorld: Agentic Novel-to-Film Generation through Dynamic Cinematic World Modeling,” *arXiv preprint arXiv:2607.19038*, 2026.
- [359] Q. Li *et al.*, “PhysAgent: Reflective Agentic Physics Control for Physically Plausible Video Generation,” *arXiv preprint arXiv:2607.16355*, 2026.
- [360] A. Tyagi *et al.*, “Beyond Trial-and-Error: Agentic Optimization for Image-to-Video Adherence,” *arXiv preprint arXiv:2608.12290*, 2026.
- [361] M. Li *et al.*, “ETPDesigner: Multi-Agent Orchestration for Interactive Multimodal Electronic Theater Program,” *arXiv preprint arXiv:2607.19947*, 2026.
- [362] M. Xu *et al.*, “WebVIA: A Web-based Vision-Language Agentic Framework for Interactive and Verifiable UI-to-Code Generation,” *arXiv preprint arXiv:2511.06251*, 2025.
- [363] S. Zhao *et al.*, “CutClaw: Agentic Hours-Long Video Editing via Music Synchronization,” *arXiv preprint arXiv:2603.29664*, 2026.
- [364] X. Tang *et al.*, “AutoMV: An Automatic Multi-Agent System for Music Video Generation,” *arXiv preprint arXiv:2512.12196*, 2025.
- [365] T. Xie *et al.*, “PhysCodeBench: Benchmarking Physics-Aware Symbolic Simulation of 3D Scenes via Self-Corrective Multi-Agent Refinement,” *arXiv preprint arXiv:2604.23580*, 2026.
- [366] H. Zhang *et al.*, “GS-Agent: Creating 4D Physical Worlds With Generative Simulation,” *arXiv preprint arXiv:2607.21522*, 2026.
- [367] Z. Hu *et al.*, “SceneCraft: An LLM agent for synthesizing 3D scene as Blender code,” in *Proc. International Conference on Machine Learning*, 2024.
- [368] J. Wei *et al.*, “VideoWeaver: Evaluating and Evolving Skills for Agentic Long Video Generation,” *arXiv preprint arXiv:2606.08091*, 2026.
- [369] X. Yin, X. Wang, L. Pan, L. Li, X. Wan, and W. Y. Wang, “Gödel Agent: A Self-Referential Agent Framework for Recursively Self-Improvement,” in *Proc. ACL*, 2025, , pp. 27890–27913.
- [370] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, “Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents,” in *Proc. ICLR*, 2026.

- [371] H. Kang *et al.*, “SimWorld Studio: Automatic Environment Generation with Evolving Coding Agent for Embodied Agent Learning,” *arXiv preprint arXiv:2605.09423*, 2026.
- [372] C. Li *et al.*, “AVA-Encoder: Towards Agent-Native Video Representation Learning,” *arXiv preprint arXiv:2608.12313*, 2026.
- [373] J. Liu *et al.*, “SymbOmni: Evolving Agentic Omni Models via Symbolic Concept Learning,” *arXiv preprint arXiv:2607.12042*, 2026.
- [374] X. Zhang *et al.*, “Generative Universal Verifier as Multimodal Meta-Reasoner,” *arXiv preprint arXiv:2510.13804*, 2025.
- [375] R. Huang *et al.*, “AlphaGRPO: Unlocking Self-Reflective Multimodal Generation in UMMs via Decompositional Verifiable Reward,” *arXiv preprint arXiv:2605.12495*, 2026.
- [376] O. Nabati *et al.*, “Preference Adaptive and Sequential Text-to-Image Generation,” *arXiv preprint arXiv:2412.10419*, 2024.
- [377] U. Gadot *et al.*, “Policy Optimized Text-to-Image Pipeline Design,” *arXiv preprint arXiv:2505.21478*, 2025.
- [378] R. Tian *et al.*, “UniGen: Enhanced Training & Test-Time Strategies for Unified Multimodal Understanding and Generation,” *arXiv preprint arXiv:2505.14682*, 2025.
- [379] T. Ren *et al.*, “SCOPE: Structured Decomposition and Conditional Skill Orchestration for Complex Image Generation,” *arXiv preprint arXiv:2605.08043*, 2026.
- [380] Y. Zhou *et al.*, “Draw ALL Your Imagine: A Holistic Benchmark and Agent Framework for Complex Instruction-based Image Generation,” *arXiv preprint arXiv:2505.24787*, 2025.
- [381] W. Xie *et al.*, “MME-Unify: A Comprehensive Benchmark for Unified Multimodal Understanding and Generation Models,” *arXiv preprint arXiv:2504.03641*, 2025.
- [382] H. Luo *et al.*, “AtelierEval: Agentic Evaluation of Humans & LLMs as Text-to-Image Promoters,” *arXiv preprint arXiv:2605.22645*, 2026.
- [383] X. Liu *et al.*, “AIGVE-MACS: Unified Multi-Aspect Commenting and Scoring Model for AI-Generated Video Evaluation,” *arXiv preprint arXiv:2507.01255*, 2025.
- [384] Z. Huang *et al.*, “PPTBench: Towards Holistic Evaluation of Large Language Models for PowerPoint Layout and Design Understanding,” *arXiv preprint arXiv:2512.02624*, 2025.
- [385] X.-S. Chen *et al.*, “PresentBench: A Fine-Grained Rubric-Based Benchmark for Slide Generation,” *arXiv preprint arXiv:2603.07244*, 2026.
- [386] A. Gandhi *et al.*, “PPT-Eval: A Benchmark for Computer-Use Agents on PowerPoint Tasks,” *arXiv preprint arXiv:2606.31154*, 2026.
- [387] Z. He *et al.*, “Vision2Web: A Hierarchical Benchmark for Visual Website Development with Agent Verification,” *arXiv preprint arXiv:2603.26648*, 2026.
- [388] Y. Huang, S. Tu, X. Yan, Z. Yang, J. Han, X. Hang, Y.-G. Jiang, and Z. Wu, “Va-judger: Reward modeling from human preference feedback for joint video-audio generation,” *arXiv preprint arXiv:2608.18607*, 2026.
- [389] S. Kim, G. Park, S. Kim, and S.-H. Nam, “VisAgent: Narrative-preserving story visualization framework,” *arXiv preprint arXiv:2503.02399*, 2025.
- [390] Z. Yang *et al.*, “Scaling large reasoning models beyond human supervision: A path toward superintelligence,” *arXiv preprint arXiv:2608.31075*, 2026.